Computers & Security 172 (2027) 105133

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

MSTE-CAN: Multi-channel Spatial–Temporal Encoding and Coordinate Attention-based ResNet for high-efficiency CAN bus intrusion detection

Luofei Jia *, Jian Zhang, Lingxuan Li, LingYun Yan, Dafei Lin, Zhongyi Zhou

School of Computer Engineering and Cybersecurity, Guang'an Institute of Technology, Guang'an, Sichuan, China

ARTICLE INFO

Dataset link: https://ocslab.hksecurity.net/Datasets/CAN-intrusion-dataset, https://0cs-github.github.io/ROAD-Dataset/, https://sites.google.com/view/hc-rl/

Keywords:
Automotive cybersecurity
Intrusion Detection System (IDS)
Controller Area Network (CAN)
Spatial–temporal encoding
Coordinate attention
Real-time edge computing

ABSTRACT

The security of the Controller Area Network (CAN) bus is fundamental to the functional safety of modern intelligent connected vehicles. However, existing intrusion detection systems (IDSs) still face a fundamental trade-off between high-precision detection of subtle spoofing attacks and the strict real-time processing requirements of resource-constrained automotive electronic control units (ECUs). This paper presents MSTE-CAN, a high-efficiency IDS framework that synergizes a novel Multi-channel Spatial–Temporal Encoding (MSTE) mechanism with a lightweight Coordinate Attention (CA)-augmented ResNet-18 architecture. The MSTE module transforms discrete CAN message sequences into high-fidelity three-channel tensors that expose latent attack signatures across complementary physical dimensions—spatial signal states, temporal dynamics, and statistical deviations—which are often obscured in traditional 1D sequence representations. The CA module, placed exclusively in the shallowest network layer, provides precise coordinate-level anomaly localization with minimal computational overhead. This **encoding-attention synergy**–where MSTE makes fine-grained attack signatures structurally visible in a multi-dimensional representation and CA anchors spatially at their exact coordinates—constitutes the core technical advancement.

Benchmarked on the public Car-Hacking-Dataset, MSTE-CAN achieves 99.96% accuracy and a macro F1-score of 0.9989, with a low false positive rate of 0.0295% and false negative rate of 0.0631%. Cross-dataset evaluation on two independent vehicle platforms—the ROAD Dataset (dynamometer testbed) and the HCRL Survival Analysis Dataset (Sonata and Spark vehicles)–demonstrates the method's cross-platform transferability: when independently trained on ROAD with the identical architecture and hyperparameters, MSTE-CAN achieves 95.85% accuracy and 0.9281 ROC-AUC (macro F1 of 0.7928); under cross-dataset transfer from Car-Hacking to HCRL (with target-domain normalization fitted on the HCRL training partition), MSTE-CAN achieves a binary F1-score of 0.9328 with system-wide FNR of 0.19%. Per-class analysis identifies physical-domain attacks and platform-specific normal-traffic characteristics as the primary sources of cross-platform performance variation, with MSTE-encoded attack signatures exhibiting strong cross-platform consistency. The framework delivers an inference time of 1.1341 ms, aligning with the low-latency prerequisites of edge-side in-vehicle monitoring. Ablation studies validate the contribution of each MSTE channel and empirically establish shallow-layer CA placement as the optimal configuration, positioning MSTE-CAN as a practical and competitive solution for edge-side automotive intrusion detection.

1. Introduction

With the rapid advancement of Intelligent Connected Vehicles (ICVs), modern automobiles have transitioned from isolated mechanical entities into sophisticated, interconnected mobile computing platforms. This paradigm shift is catalyzed by the deep integration of Advanced Driver Assistance Systems (ADAS), Over-the-Air (OTA) updates, and Vehicle-to-Everything (V2X) communications. At the core of this interconnected architecture lies the Controller Area Network (CAN) bus, a robust broadcast protocol that facilitates real-time communication between hundreds of Electronic Control Units (ECUs) (Miller and Valasek, 2015; Navet et al., 2005). Despite its industrial-grade reliability, the CAN protocol was engineered in an era devoid of external connectivity. Consequently, it lacks fundamental security primitives—such as message encryption, sender authentication, and integrity verification—leaving it inherently vulnerable to modern cyber threats (Bozdal et al., 2018; Wu et al., 2019; Koscher et al., 2010).

* Corresponding author.
E-mail addresses: 22025332@gaitpo.edu.cn (L. Jia), 12025102@gaitpo.edu.cn (J. Zhang), 22025267@gaitpo.edu.cn (L. Li), 22025269@gaitpo.edu.cn (L. Yan), 22025334@gaitpo.edu.cn (D. Lin), 22025317@gaitpo.edu.cn (Z. Zhou).

https://doi.org/10.1016/j.cose.2026.105133
Received 27 April 2026; Received in revised form 8 August 2026; Accepted 17 August 2026
Available online 1 September 2026
0167-4048/© 2026 Published by Elsevier Ltd.

These structural vulnerabilities expand the vehicular attack surface significantly. Remote adversaries can exploit diverse entry points, including On-Board Diagnostics II (OBD-II) ports, Infotainment systems, and wireless telematics, to inject malicious traffic (Checkoway et al., 2011). High-priority **Denial of Service (DoS)** attacks can paralyze bus communication, while **spoofing attacks** can manipulate safety-critical functions, such as steering and braking, by forging sensor data. The repercussions of such intrusions range from compromised user privacy to life-threatening physical safety risks (Agrawal et al., 2022).

To fortify vehicular networks, various Intrusion Detection Systems (IDSs) have been developed. While early solutions relied on rule-based or statistical heuristics, they often struggle to detect increasingly sophisticated, low-intensity "stealthy" attacks. Recent research has pivoted toward Deep Learning (DL), leveraging Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to extract complex patterns from raw traffic. Nevertheless, two critical bottlenecks persist in the current landscape:

1. **Data Representation Efficiency:** Raw CAN data is fundamentally a one-dimensional (1D) discrete sequence. Conventional mapping techniques, such as simple grayscale conversion, frequently fail to preserve the high-dimensional correlations between protocol semantics and temporal dynamics.
2. **Real-time Deployment Constraints:** High-precision DL models are often computationally prohibitive, exceeding the stringent memory and processing budgets of automotive-grade ECUs. In a high-speed CAN environment (500 kbps to 1 Mbps), a detection latency of even a few milliseconds can render a defense mechanism obsolete before corrective action can be taken.

To bridge these gaps, this paper proposes **MSTE-CAN**, a high-efficiency IDS framework tailored for ICVs. We introduce a new **Multi-channel Spatial–Temporal Encoding (MSTE)** mechanism that synthesizes raw CAN signals into 2D RGB-like images. By encoding three complementary physical dimensions of the bus traffic–spatial magnitude, temporal variation, and statistical deviation–into distinct visual channels, this representation explicitly exposes latent anomalies that traditional 1D sequences might obscure.

While prior works have explored converting CAN traffic into images for CNN-based detection (Song et al., 2020), existing approaches predominantly rely on single-channel (grayscale) mappings that collapse protocol semantics and temporal dynamics into a unified intensity representation. Such representations discard the multi-dimensional structure of CAN traffic and limit the model's capacity to disentangle spatial signal patterns from temporal volatility. MSTE-CAN advances beyond these approaches through three tightly coupled design choices. First, the three-channel MSTE encoding (detailed in Section 3.1) decomposes CAN traffic into orthogonal physical dimensions, each contributing complementary discriminative information. Second, the **nearest-neighbor upsampling** strategy deliberately preserves the discrete boundaries of CAN protocol fields (CAN ID and data bytes), unlike bilinear or bicubic interpolation used in prior work that introduces semantic blurring. Third, the **Coordinate Attention (CA)** module is placed in the shallowest ResNet layer, where full spatial resolution is retained, enabling byte-level attack localization that global attention mechanisms (e.g., SE, CBAM) and deeper-layer placement cannot achieve. This encoding-attention synergy constitutes the core technical contribution.

Furthermore, we design a lightweight **CA-ResNet-18** architecture. Unlike standard global attention, the CA module embeds spatial coordinates into channel attention, enabling the backbone to localize specific message fields where attacks manifest. Optimized with AdamW and Dropout regularization, the system achieves a favorable balance between classification precision and execution speed.

The primary contributions of this work are as follows:

- **Multi-Channel Visual Encoding:** We propose the MSTE scheme, which transforms 1D CAN traffic into multi-channel visual representations, enhancing the discriminability of subtle spoofing patterns.
- **Coordinate-Sensitive Backbone:** We develop a CA-ResNet-18 network that utilizes Coordinate Attention to capture long-range dependencies across both temporal and spatial axes while maintaining a lightweight computational footprint.
- **Rigorous Empirical Validation:** Extensive evaluations on the Car-Hacking-Dataset validate that MSTE-CAN achieves high detection accuracy with low latency, meeting the stringent real-time requirements of vehicular environments. Cross-dataset validation on the ROAD Dataset and the HCRL Survival Analysis Dataset demonstrates the method's cross-platform transferability and quantifies the performance boundaries for physical-domain attacks. The HCRL evaluation further isolates normal-traffic domain shift as the dominant source of cross-platform performance variation, confirming that MSTE-encoded attack signatures exhibit strong cross-platform consistency.

The remainder of this paper is organized as follows: Section 2 reviews the theoretical background and related work. Section 3 elaborates on the proposed MSTE encoding scheme and the CA-ResNet-18 architecture. Section 4 describes the experimental setup, including dataset characteristics and evaluation metrics. Section 5 presents the experimental results and provides a comprehensive discussion of the performance. Finally, Section 6 concludes the paper and discusses future research directions.

2. Background and related work

The evolution of in-vehicle network security has progressed from rudimentary specification-based and statistical monitoring (Larson et al., 2008; Müter and Asaj, 2011) to advanced deep learning-based detection. Current literature can be categorized into three primary paradigms: conventional machine learning and optimization, sequence-based deep learning, and hybrid multi-dimensional fusion architectures.

2.1. Conventional machine learning and heuristic optimization

Early IDS research concentrated on physical layer fingerprints and message frequency stability. Taylor et al. (2015) utilized Support Vector Machines (SVM) to flag attacks by identifying deviations in CAN packet arrival intervals. Garg et al. (2019) introduced "Sec-IoV", a multi-stage anomaly detection model utilizing a hybrid optimizer based on Artificial Bee Colony (ABC) and Cauchy distribution functions to solve multi-objective optimization challenges. Building on metaheuristic concepts, Ileri et al. (2025) proposed "MetaCAN", which leverages an adaptive hybrid of Particle Swarm Optimization (PSO) and Cuckoo Search (CS) to enhance the global search capabilities and convergence speed of traditional classifiers. Additionally, the work of Sami et al. (2020), Rosay et al. (2022) established the hardware-level feasibility of MLP-based architectures on automotive microprocessors, emphasizing the necessity of real-time edge execution.

2.2. Deep learning for spatial and temporal modeling

As attack vectors grew more complex, research shifted toward deep learning for non-linear feature extraction. Hossain et al. (2020a) demonstrated the efficacy of 1D Convolutional Neural Networks (CNN) for vehicular anomaly detection, while Song et al. (2020) adapted 2D CNNs to better capture the structural nuances of CAN message payloads. Hossain et al. (2020b) further proposed an LSTM-based architecture that leverages sequential message dependencies for intrusion classification. To address the resource constraints of embedded systems,

Luo et al. (2025) proposed a lightweight model utilizing spectral residuals for redundancy removal and depth-wise separable convolutions, specifically optimized for time-critical automotive environments.

A notable sub-direction within CNN-based CAN IDS is the transformation of raw CAN message streams into 2D image-like representations, enabling the application of mature computer vision backbones. Song et al. (2020) pioneered this approach by mapping CAN IDs and payloads to pixel intensities, but their single-channel encoding collapses multi-dimensional protocol semantics (CAN ID priority, data byte values, inter-message timing) into a unified grayscale representation, discarding discriminative information. Subsequent works have largely followed this single-channel paradigm without systematically exploring multi-dimensional encoding strategies. Moreover, the common use of bilinear or bicubic interpolation for upsampling introduces semantic blurring across CAN field boundaries, which is detrimental to byte-level anomaly localization. These limitations motivate our proposed MSTE mechanism, which decomposes CAN traffic into three orthogonal physical channels and preserves discrete field boundaries through nearest-neighbor upsampling.

To account for the sequential dependencies in CAN traffic, recurrent architectures gained prominence. Taylor et al. (2016) employed Long Short-Term Memory (LSTM) units to predict subsequent packets, using log-loss as an anomaly signal. Ding et al. (2022) combined Bidirectional LSTM (Bi-LSTM) with a sliding window strategy to capture bidirectional temporal correlations. More recently, Yin et al. (2025) proposed a temporal–spatial fusion framework to identify highly concealed attacks, such as Same Origin Method Execution (SOME), by integrating local spatial motifs with global temporal trends.

2.3. Hybrid models and multi-dimensional fusion

The current frontier of IDS research involves fusing multiple modalities or structural abstractions. Islam et al. (2020) introduced the first graph-based defense system for CAN, which employs a four-stage process using the chi-squared method to analyze relational structures between network nodes. Similarly, Kang et al. (2024) developed "CANival", a multimodal framework using unsupervised learning to adapt to evolving, zero-day vehicular threats.

The integration of attention mechanisms has further refined detection accuracy. Javed et al. (2021) proposed CANintelliIDS, combining CNN with attention-based GRU for spatial–temporal feature extraction. Wei et al. (2023) utilized Denoising Autoencoders (DAE) with attention to re-weight critical features, while Taneja and Kumar (2024) proposed the ACL-IDS model, combining CNN-LSTM with attention to spotlight significant temporal-local segments. More recently, Cheng et al. (2022) introduced TCAN-IDS with temporal convolutional attention, and Sun et al. (2024) proposed a cross-chain detection framework to address multi-source CAN traffic analysis. These trends toward robust, multi-dimensional representations align with our proposed **MSTE-CAN** framework, which explicitly encodes spatial, temporal, and statistical dimensions through a coordinate-sensitive attention mechanism.

2.4. Research gaps and motivation

Despite these advancements, three critical gaps remain. **First**, prior image-based CAN IDS methods (Song et al., 2020) rely on single-channel encodings that collapse multi-dimensional CAN protocol semantics into unified intensity representations, discarding the complementary discriminative information carried by temporal dynamics and statistical deviations. **Second**, the standard practice of bilinear or bicubic upsampling introduces semantic blurring across discrete CAN field boundaries, undermining the fine-grained spatial precision needed for byte-level anomaly localization. **Third**, many high-performing methods employ complex architectures—particularly deep attention hierarchies and large Transformer backbones (Nguyen et al., 2023; Jo and Kim, 2024)—that incur excessive computational overhead, struggling to meet the sub-10 ms real-time requirements of high-speed CAN buses. Furthermore, standard global attention mechanisms (e.g., SE, CBAM) operate on channel-wise or spatially-pooled feature statistics, ignoring the precise coordinate-level importance of individual message fields in a 2D-mapped representation. These limitations motivate the development of **MSTE-CAN**, which addresses all three gaps through: (i) a three-channel spatial–temporal-statistical encoding that preserves complementary discriminative dimensions, (ii) nearest-neighbor upsampling that maintains discrete field boundaries, and (iii) a shallow-layer coordinate attention mechanism that provides fine-grained spatial sensitivity with minimal computational overhead.

3. Methodology

The proposed MSTE-CAN framework consists of two primary stages: (1) a Multi-channel Spatial–Temporal Encoding (MSTE) module that transforms raw 1D CAN data into 2D multi-channel feature maps, and (2) a lightweight CA-ResNet-18 classifier that integrates Coordinate Attention (CA) for high-precision intrusion detection. The overall system architecture and modular data flow are illustrated in Fig. 1.

3.1. Multi-channel spatial–temporal encoding (MSTE)

To bridge the gap between discrete, one-dimensional automotive network traffic and the high-capacity feature extraction of deep convolutional networks, we propose the Multi-channel Spatial–Temporal Encoding (MSTE) mechanism. This method transforms a sequence of L consecutive CAN messages into a high-fidelity three-channel RGB tensor, effectively preserving both the physical signal states and temporal dynamics of the bus.

Given a sliding window $\mathbf{W} \in \mathbb{R}^{L\times 9}$ comprising L consecutive messages—each containing a CAN ID and 8 data bytes—the MSTE mechanism constructs a multi-dimensional representation $\mathbf{X} \in \mathbb{R}^{L\times 9\times 3}$. Each channel is meticulously designed to encode a distinct physical dimension of vehicular traffic:

1. **Spatial State Channel (R):** This channel represents the normalized magnitude of CAN signals. For each message index $t \in \{1,\dots,L\}$ and feature index $f \in \{1,\dots,9\}$, the value $R_{t,f}$ is normalized using the 1st and 99th percentiles of the training dataset to mitigate extreme outliers. This channel provides the model with the absolute "spatial" layout of message payloads within the observed window.
2. **Temporal Difference Channel (G):** To emphasize rapid signal fluctuations—a characteristic byproduct of injection attacks—the G channel encodes the absolute first-order temporal difference:

$$G_{t,f} = |R_{t,f} - R_{t-1,f}|, \quad \text{with } G_{1,f} = 0 \tag{1}$$

   By capturing signal transition volatility, this channel makes high-frequency anomalies (e.g., DoS flooding) visually distinct from the stable patterns of normal vehicular operation.
3. **Statistical Offset Channel (B):** To identify stealthy attacks that maintain values within a "normal" range but disrupt local statistical consistency, the B channel records deviations from the window's mean μ_f:

$$B_{t,f} = |R_{t,f} - \mu_f|, \quad \text{where } \mu_f = \frac{1}{L}\sum_{i=1}^{L} R_{i,f} \tag{2}$$

   This provides a contextual "anomaly score" for each feature in the local temporal neighborhood, significantly enhancing detection of fine-grained spoofing attacks such as Gear or RPM manipulation.

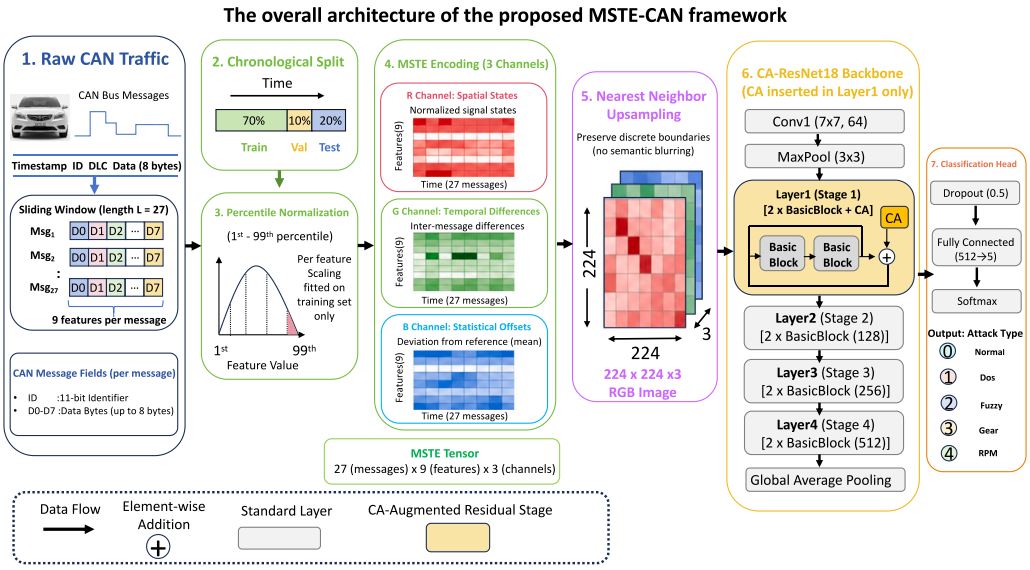


Fig. 1. The overall architecture of the proposed MSTE-CAN framework. The pipeline consists of seven stages: (1) Raw CAN traffic acquisition via sliding window; (2) Chronological data partitioning; (3) Percentile normalization; (4) MSTE encoding into three-channel R/G/B tensors capturing spatial state, temporal difference, and statistical offset; (5) Nearest-neighbor upsampling to preserve field boundaries; (6) Feature extraction via CA-ResNet-18 with Coordinate Attention; and (7) Classification head outputting predictions across five attack/normal classes.

The resulting $L \times 9 \times 3$ tensor is upsampled to a fixed resolution of 224×224 pixels using **nearest-neighbor interpolation**. This method is deliberately chosen over bilinear or bicubic alternatives to prevent semantic blurring of discrete CAN ID boundaries and data byte transitions. By preserving sharp feature edges, MSTE encoding ensures that subsequent convolutional filters can accurately capture the coordinate-level signatures of malicious intrusions.

3.2. Visual representation analysis and interpretability

A core advantage of the MSTE mechanism is its capability to translate abstract, low-level CAN traffic into high-level interpretable visual signatures. As illustrated in Fig. 2, different attack types exhibit distinct morphological and chromatic patterns, which directly correspond to the physical signal attributes encoded in the three RGB channels. This inherent interpretability is critical for automotive safety systems, as it enables engineers to validate and debug detection decisions.

The MSTE encoding provides a robust basis for model interpretability:

- **Normal Operation:** These images display consistent textures and smooth transitions, reflecting the periodic and deterministic nature of benign CAN message sequences.
- **High-Intensity Attacks (DoS and Fuzzy):** These intrusions introduce significant temporal and spatial entropy. In DoS attacks, the rapid injection of conflicting CAN IDs causes extreme volatility in the **Green (Temporal Difference)** and **Blue (Statistical Offset)** channels, appearing as dense, high-contrast horizontal lines. Fuzzy attacks, characterized by randomized payloads, result in disorganized pixel distributions across all three channels.
- **Subtle Spoofing (RPM and Gear):** These are the most deceptive attacks, as they only modify specific data bytes while preserving normal transmission frequencies. MSTE captures these as discrete "color shifts" or "broken textures" localized within specific columns (corresponding to individual CAN ID and data byte fields).

This spatial localization of attack signatures aligns directly with the design of the **CA** module, whose directional positional encoding guides the backbone to focus on the exact coordinates where anomalies occur rather than processing the entire feature map uniformly.

3.3. Coordinate Attention (CA) mechanism

In vehicular network security, both the specific feature patterns (e.g., bit-flips in data fields) and their temporal dependencies are critical for accurate intrusion detection. To capture these nuances, we integrate the **Coordinate Attention (CA)** mechanism (Hou et al., 2021). Unlike standard **Squeeze-and-Excitation (SE)** blocks that only aggregate global channel-wise information, CA embeds explicit positional information into channel attention, enabling the model to capture long-range dependencies along one spatial direction while preserving precise positional awareness along the other.

As illustrated in Fig. 3, for an input feature map $\mathbf{F} \in \mathbb{R}^{C\times H\times W}$, CA first performs global average pooling along the horizontal and vertical directions separately to generate direction-aware feature maps:

$$z_c^h(h) = \frac{1}{W} \sum_{0\le i<W} F_c(h, i) \tag{3}$$

$$z_c^w(w) = \frac{1}{H} \sum_{0\le j<H} F_c(j, w) \tag{4}$$

In the **MSTE-CAN** framework, these two directional encodings correspond to the **temporal sequence** (h direction) and **entity features** (w direction) of CAN traffic. Following the pooling stage, the two maps are concatenated and transformed through a shared 1×1 convolutional layer $\mathcal{F}_1$ and a non-linear activation δ to yield an intermediate feature map $\mathbf{f} \in \mathbb{R}^{C/r\times(H+W)}$:

$$\mathbf{f} = \delta(\mathcal{F}_1([\mathbf{z}^h, \mathbf{z}^w])) \tag{5}$$

The tensor $\mathbf{f}$ is then split into two separate tensors and transformed via 1×1 convolutions to yield the final attention weights $\mathbf{g}^h$ and $\mathbf{g}^w$. This dual-axis mechanism enables the model to precisely locate **where** (specific data byte) and **when** (specific message index within the sliding window) an anomaly occurs.

3.4. CA-ResNet-18 architecture

The overall architecture is built upon a modified **ResNet-18** backbone optimized for lightweight in-vehicle deployment. The core innovation lies in the integration of CA modules into the residual blocks. As detailed in Fig. 3, the CA module recalibrates feature maps within the

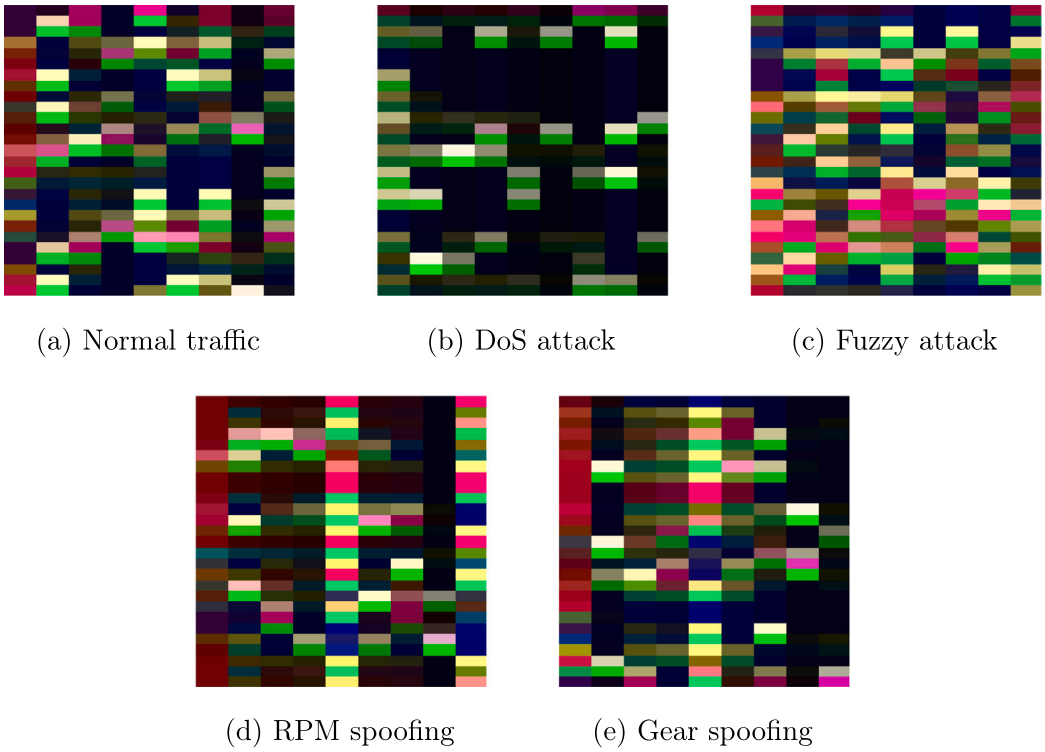


Fig. 2. Visual signatures of CAN traffic under MSTE encoding. Normal traffic exhibits stable, repetitive color blocks; DoS attacks manifest as intense horizontal stripes due to high-frequency injections; Fuzzy attacks present chaotic salt-and-pepper noise patterns; Spoofing attacks show localized color shifts in specific feature columns while maintaining an overall stable structure. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

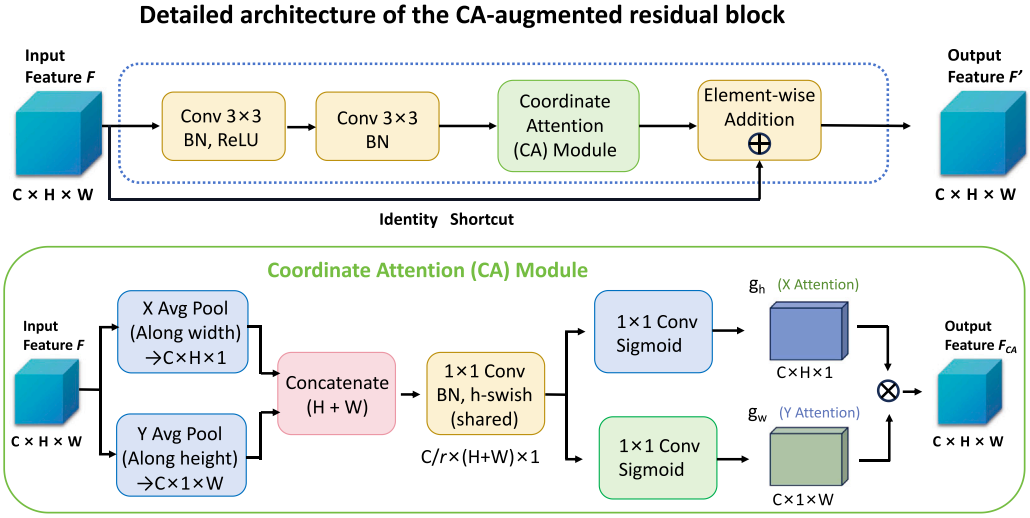


Fig. 3. Detailed architecture of the CA-augmented residual block.

skip-connection structure, enhancing salient spatial–temporal patterns while suppressing irrelevant background noise.

The network processes the 224 × 224 MSTE-encoded tensors through an initial 7 × 7 convolution layer, followed by four stages of CA-augmented residual blocks. Based on extensive ablation studies (Section 5.4), we strategically place the CA module **only in the first stage (CA-Layer 1)**. This configuration is chosen because Layer 1 maintains the highest spatial resolution; at this stage, CA can precisely anchor fine-grained, localized attack signatures before they are smoothed by subsequent pooling operations. Our experiments indicate that deeper CA integration leads to feature redundancy and unnecessary computational overhead without significant precision gains.

The classification head consists of a global average pooling layer and a fully connected (FC) layer with a **Dropout** rate of $p = 0.5$. The model is trained using the **AdamW** optimizer, minimizing a **Cross-Entropy** loss function with **L2** regularization:

$$\mathcal{L} = -\sum_{i=1}^{K} y_i \log(\hat{y}_i) + \lambda \|\theta\|^2 \tag{6}$$

where $\hat{y}_i$ is the predicted probability for class i, y_i is the ground truth, and λ represents the weight decay factor. This optimized architecture achieves a low computational footprint, ensuring a favorable balance between detection precision and real-time inference latency.

Table 1
Composition and statistical distribution of the experimental CAN dataset. Each attack file contains both injected (T) and background (R) messages.

Message source	Total messages	Injected (T)	Proportion (%)
Normal Traffic	928,136	0	6.31
DoS Attack	1,040,624	233,004	7.08
Fuzzy Attack	3,751,024	491,847	25.51
Gear Spoofing	4,402,977	597,252	29.95
RPM Spoofing	4,580,226	654,897	31.15
Total	14,702,987	1,977,000	**100.00**

Table 2
Composition of the ROAD test set used for cross-dataset evaluation.

Class	Window count	Proportion (%)
Accelerator	6138	2.93
Correlated	1016	0.49
Fabrication	4737	2.26
Fuzzing	117	0.06
Masquerade	5526	2.64
Normal	191,685	91.62
Total	**209,219**	**100.00**

4. Experimental evaluation

4.1. Dataset description and statistics

To evaluate the performance of the proposed **MSTE-CAN** framework, we utilize the **Car-Hacking-Dataset** provided by the Hacking and Countermeasure Research Lab (HCRL) (Song et al., 2020). This dataset is widely recognized as a benchmark for vehicular intrusion detection, containing real-world CAN bus traffic captured via the OBD-II port of a Kia Soul vehicle during active driving.

The dataset encompasses a complete collection of benign driving sequences and four representative malicious injection scenarios:

- **DoS Attack:** Injecting high-priority messages at extremely high frequencies to occupy bus bandwidth.
- **Fuzzy Attack:** Injecting messages with randomized CAN IDs and data payloads to cause unpredictable vehicle behavior.
- **Gear/RPM Spoofing:** Injecting forged packets targeting specific CAN IDs to manipulate the dashboard display or engine control logic.

Each CAN message record is characterized by a timestamp, an 11-bit CAN ID, a Data Length Code (DLC), and an 8-byte data payload. Although the DLC field is present in the raw dataset, it is used solely for message filtering (retaining only DLC=8 messages to maintain a fixed 8-byte payload width) and is not included as an input feature to the MSTE encoder. The input feature vector for each message is therefore 9-dimensional (1 CAN ID + 8 data bytes). In our study, we process the entire corpus of **14,702,987** messages, of which 1,977,000 are injected attack messages (T-type) and 11,797,851 are background traffic (R-type), with the remaining 928,136 messages constituting pure normal driving data. Each attack-containing file includes both injected and background messages. The detailed composition and class distribution of the dataset are summarized in Table 1.

4.1.1. ROAD dataset: Cross-dataset validation

To address the reviewer's concern regarding single-dataset evaluation and to validate the generalizability of MSTE-CAN, we introduce the **ROAD Dataset** as an independent cross-dataset benchmark. The ROAD (Real ORNL Automotive Dynamometer) Dataset (Verma et al., 2024) was collected at Oak Ridge National Laboratory from a vehicle dynamometer testbed, representing a fundamentally different vehicular platform from the Kia Soul used in the Car-Hacking-Dataset. The dataset encompasses approximately 28.2 million CAN messages spanning six classes:

- **Normal:** Benign CAN traffic captured during ambient driving conditions.
- **Fuzzing:** Random CAN ID and payload injection attacks analogous to Fuzzy attacks in Car-Hacking.
- **Fabrication:** Targeted spoofing of specific CAN IDs (speedometer, coolant, and reverse light signals).
- **Masquerade:** A sophisticated attack variant where the adversary suppresses legitimate ECU messages and replaces them with malicious payloads, making it particularly difficult to detect.
- **Correlated Signal Attack:** Coordinated manipulation of multiple interdependent signals to deceive vehicle control logic.
- **Accelerator Attack:** A physical attack in which the accelerator pedal position is mechanically tampered with. Critically, this attack introduces *no CAN-level message injection*, with all CAN traffic reflecting legitimate pedal position readings corresponding to the physical tampering.

The raw CAN traffic is processed using the *identical* MSTE encoding pipeline described in Section 3.1, with no parameter adjustments or architectural modifications. Attack intervals are labeled using the official `capture_metadata.json` files, and the data is split chronologically (70/10/20) following the same temporal partitioning protocol. After MSTE encoding, the ROAD dataset yields **1,046,090** image windows (Train: 732,263; Val: 104,608; Test: 209,219). The detailed composition of the test set is summarized in Table 2.

4.1.2. HCRL survival analysis dataset: Cross-vehicle cross-attack validation

To further isolate the sources of cross-dataset performance variation identified in the ROAD evaluation, and to test generalization across both vehicle platforms and attack types simultaneously, we introduce the **HCRL Survival Analysis Dataset for Automobile IDS** (Hacking and Countermeasure Research Lab (HCRL), 2019). This dataset contains real-world CAN bus traffic recorded from three Hyundai-Kia vehicles—Sonata, Soul, and Spark—during active driving under both normal conditions and three categories of CAN-level injection attacks:

- **Flooding:** High-frequency injection of CAN messages with a specific CAN ID to overwhelm the bus, analogous to DoS attacks in the Car-Hacking-Dataset.
- **Fuzzy:** Injection of messages with randomized CAN IDs and data payloads to disrupt normal ECU communication, directly corresponding to Fuzzy attacks in Car-Hacking.
- **Malfunction:** Targeted injection of spoofed messages simulating ECU malfunction, analogous to Gear/RPM spoofing in Car-Hacking.

Critically, to avoid same-vehicle overlap, we **exclude all HCRL Kia Soul data** (the same vehicle model as the Car-Hacking-Dataset) and retain only the **Sonata** and **Spark** vehicles as held-out test platforms, totaling approximately 938,000 CAN messages. This deliberate exclusion ensures the evaluation represents a genuine cross-vehicle generalization test, where the model encounters entirely unfamiliar CAN ID distributions, normal traffic patterns, and vehicle-specific message periodicities.

The raw CAN traffic is processed using the *identical* MSTE encoding pipeline described in Section 3.1. Percentile-based normalization (1st and 99th percentiles) is fitted on the HCRL training partition exclusively, and data is split chronologically (70/10/20) following the same temporal partitioning protocol. Windows are labeled by the priority-based rule described in Section 4.2. After MSTE encoding, the HCRL dataset yields **34,730** image windows (Train: 24,315; Val: 3470; Test: 6945). The detailed composition of the test set is summarized in Table 3.

Table 3
Composition of the HCRL test set used for cross-vehicle cross-attack evaluation.

Class	Window count	Proportion (%)
Flooding	1235	17.78
Fuzzy	611	8.80
Malfunction	824	11.86
Normal	4275	61.56
Total	**6945**	**100.00**

4.2. Data preprocessing and MSTE configuration

To prepare raw vehicular traffic for the deep learning pipeline, we transform discrete 1D CAN sequences into 2D spatial–temporal visual representations using the proposed MSTE mechanism. The complete preprocessing workflow consists of three sequential steps:

1. **Feature Transformation and Normalization:** Raw hexadecimal CAN IDs and 8-byte data payloads are first converted into their decimal equivalents. To mitigate the impact of sensor noise and extreme outliers while *strictly preventing data leakage*, we employ **percentile-based scaling** using only the 1st and 99th percentiles of the *training dataset* distribution. Signal values are mapped into the range $[0, 1]$, ensuring the model remains robust against transient signal spikes while preserving the relative magnitude of payloads.
2. **Temporal Windowing:** The continuous stream of normalized CAN messages is segmented into discrete temporal blocks using a non-overlapping sliding window of size $L = 27$. This window size is motivated by the inherent structure of the CAN protocol: each message contains exactly 9 features (1 CAN ID + 8 data bytes), and the three-channel MSTE design requires uniform spatial dimensions across all channels. A window of 27 messages generates a consistent 27×9 feature matrix for each channel, which can be directly concatenated into a $27 \times 9 \times 3$ tensor without any padding, truncation, or information loss. This structurally motivated choice is further validated by a systematic sensitivity analysis (Section 5.5.3), which confirms that $L = 27$ is the empirical optimum across all detection metrics, with performance degrading under both context starvation ($L < 27$) and feature dilution ($L > 27$).
3. **Window Labeling and Data Leakage Prevention:** To ensure the integrity of the chronological split, we enforce strict temporal isolation at window boundaries. The non-overlapping sliding window operates independently within each data partition (train, validation, and test), meaning no window spans across partition boundaries. Each window is labeled based on a **priority-based rule**: if any message within the 27-message block falls within an attack interval (as recorded in the dataset metadata), the entire window is assigned to the corresponding attack class. Windows containing only normal messages are labeled as Normal. In the rare case where multiple attack types co-occur within a single window, the window is assigned to the attack class with the majority of messages. For the Car-Hacking-Dataset, attack and normal intervals are provided by the dataset authors with per-message ground-truth labels. For the ROAD Dataset, attack intervals are defined by the official `capture_metadata.json` files. This window-level labeling strategy is consistent with real-time deployment, where a detection decision must be made for each arriving window rather than for individual messages. The chronological 70/10/20 partitioning is applied at the message level before window construction, ensuring that messages from different temporal periods never mix within a single window. This protocol eliminates the risk of near-duplicate patterns leaking across data partitions.
4. **Spatial–Temporal Tensor Construction:** The raw $27 \times 9 \times 3$ MSTE tensors are upsampled to a fixed resolution of 224×224 pixels to match the standard input specification of the ResNet-18 backbone. We exclusively adopt nearest-neighbor interpolation for this upsampling process. In contrast to bilinear or bicubic interpolation, which produce smooth transitions between adjacent pixels by weighted averaging, nearest-neighbor interpolation preserves the discrete, step-like nature of CAN protocol fields. This prevents the semantic blurring of CAN ID boundaries and data byte transitions, which is critical for maintaining the coordinate-level localization capability of the subsequent convolutional layers and coordinate attention module, as discussed in Section 3.1.

4.3. Experimental setup and training strategy

Hardware and Software Environment: All experiments were implemented using the PyTorch 2.1 deep learning framework and executed on a high-performance workstation equipped with an **NVIDIA GeForce RTX 5090 GPU** (24 GB VRAM) and 64 GB of system memory. The operating environment was Ubuntu 22.04 LTS with CUDA 12.1 acceleration.

Model Configuration: The backbone architecture is based on ResNet-18, modified with the Coordinate Attention (CA) mechanism integrated into the residual blocks. The final fully connected layer was adjusted to output five classes, preceded by a dropout layer ($p = 0.5$) to improve generalization.

Training Protocol: Given the inherent temporal nature of CAN bus traffic, random partitioning is strictly avoided as it would introduce severe data leakage and artificially inflated performance. We thus adopt a strict temporal-order partitioning strategy based on message timestamps: the first 70% of the chronologically sorted data is used for training, the middle 10% for validation and hyperparameter tuning, and the last 20% as the independent test set to simulate real-world deployment and ensure reliable performance evaluation. The optimization process utilized the **AdamW** optimizer with an initial learning rate of 1×10^{-4}. A batch size of **128** was selected to balance GPU memory utilization and gradient stability. The model was scheduled to train for a maximum of **50 epochs**; however, an **early-stopping mechanism** with a patience of 5 epochs was employed. Training was automatically terminated when the validation loss ceased to decrease, ensuring the selection of the model weight with the best generalization capability.

5. Experimental results and discussion

In this section, we evaluate the performance of the MSTE-CAN framework using both the Car-Hacking-Dataset and the ROAD Dataset for cross-dataset validation. The evaluation focuses on training convergence, multi-class detection accuracy, cross-dataset transferability, and real-time computational efficiency.

5.1. Training dynamics and convergence

The training and validation dynamics of the MSTE-CAN framework are illustrated in Fig. 4. The convergence curves reveal that the model exhibits rapid learning efficiency within the proposed MSTE feature space. Both training and validation losses decrease sharply during the initial 5 epochs, entering a stable plateau shortly thereafter. This rapid convergence suggests that the MSTE mechanism effectively simplifies the feature extraction process by transforming abstract CAN sequences into highly discriminative visual signatures, allowing the convolutional filters to capture salient intrusion patterns early in the training phase.

Specifically, the training process achieved optimal convergence at Epoch 24, at which point the early-stopping mechanism was triggered to preserve generalization. At this stage, the validation accuracy and Macro F1-score reached their peak performance, while the Error

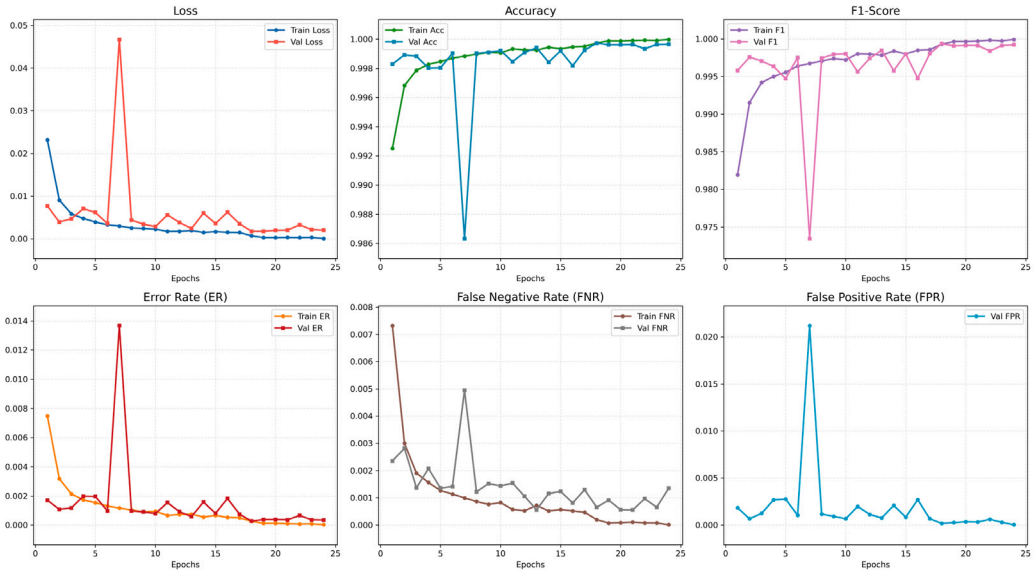


Fig. 4. Training and validation dynamics of MSTE-CAN. The subplots illustrate the convergence of Loss, Accuracy, F1-Score, Error Rate (ER), FNR, and FPR over 24 epochs.

Table 4
Detailed classification results of MSTE-CAN on the test set.

Attack type	Precision	Recall	F1-score	Accuracy	FNR	FPR
Normal	0.9996	0.9997	0.9997	0.9997	0.000295	0.000631
DoS	0.9943	0.9964	0.9953	0.9964	0.003639	0.000180
Fuzzy	0.9999	0.9996	0.9998	0.9996	0.000372	0.000010
Gear	1.0000	0.9992	0.9996	0.9992	0.000777	0.000000
RPM	1.0000	1.0000	1.0000	1.0000	0.000000	0.000000
Macro Avg	**0.9987**	**0.9991**	**0.9989**	–	–	–

Note: In multi-class classification, the class-specific FNR denotes the miss rate of samples genuinely belonging to that class, while the FPR denotes the rate at which other samples are falsely classified into that class. Following standard conventions in intrusion detection literature, the system-wide False Positive Rate (FPR) corresponds to the FNR of the Normal class (0.0295%), as it represents normal traffic misclassified as an attack. Conversely, the system-wide False Negative Rate (FNR) corresponds to the FPR of the Normal class (0.0631%), representing actual attacks misclassified as normal traffic. Optimal configuration: CA module integrated into the first stage of ResNet-18.

Rate (ER) remained consistently minimal. Crucially, as shown in the FNR and FPR subplots, the framework maintains near-zero false rates throughout the stable phase. In the context of automotive security, the consistently low **False Negative Rate (FNR)** is particularly significant, as it demonstrates the model's reliability in detecting critical injection attacks without compromise. The high degree of overlap between the training and validation curves further confirms that the **CA-ResNet-18** architecture is robust against overfitting, effectively capturing the underlying spatial–temporal dependencies of the vehicular bus traffic rather than memorizing noise.

5.2. Detection performance analysis

The overall performance of **MSTE-CAN** on the held-out independent test set is evaluated through a comprehensive suite of rigorous metrics. As summarized in Table 4, the framework achieves a test accuracy of 99.96% and a macro F1-score of 0.9989. The statistical reliability of the classifier is further validated by a **Matthews Correlation Coefficient (MCC)** of 0.9993 and a **Kappa coefficient** of 0.9993. These values, approaching the theoretical limit of 1.0, confirm strong detection consistency and demonstrate that the model's predictions substantially outperform random chance across all attack categories, even under severe class imbalance.

Class-specific metrics reveal the granular efficacy of MSTE-CAN against diverse injection vectors. Notably, the model achieves perfect precision and recall for RPM spoofing attacks, indicating complete separability between these malicious patterns and normal traffic. For other stealthy attacks including Gear spoofing and Fuzzy attacks, the F1-scores reach 0.9996 and 0.9998 respectively, showcasing the synergistic advantage of MSTE encoding and coordinate attention in capturing subtle byte-level spatial–temporal anomalies. Even under high-load DoS flooding scenarios, the model maintains a robust recall of 99.64%, ensuring that the vast majority of denial-of-service attempts are intercepted.

From a functional safety perspective, the system-wide overall false positive rate (FPR) is minimized to 0.0295%. By definition, this corresponds to the fraction of normal traffic samples misclassified as attacks, which is the most critical metric for in-vehicle deployment. This low false alarm rate helps prevent unnecessary fail-safe interventions that could disrupt legitimate ECU communications and compromise vehicle availability.

The classification performance is further visualized in the confusion matrix (Fig. 5). The matrix shows strong diagonal alignment, with only 20 out of 67,683 normal samples misidentified as attacks, and all injection attacks captured with near-total sensitivity. These results confirm that MSTE-CAN successfully translates the physical temporal-entity characteristics of CAN messages into highly discriminative visual representations.

5.3. Computational efficiency and real-time feasibility

While achieving high detection accuracy, MSTE-CAN also offers favorable computational characteristics for edge-side in-vehicle deployment. The lightweight architecture has only 11.18 million parameters

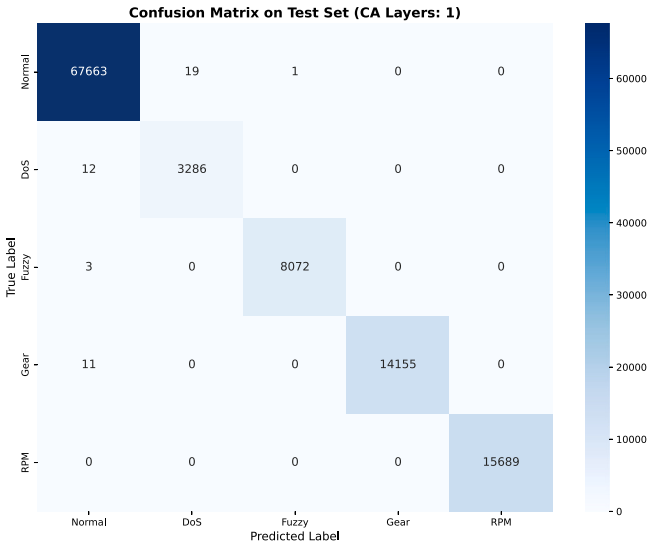


Fig. 5. Confusion matrix of MSTE-CAN on the independent test set.

and a computational complexity of 1.82 GFLOPs, representing a favorable balance between high-dimensional feature extraction and deployment feasibility on resource-constrained edge platforms. Benchmarks conducted on an NVIDIA GeForce RTX 5090 GPU yield an average pure model inference time of 1.1341 ms per window, with even the 99th percentile remaining below 1.24 ms. This corresponds to a pure model inference throughput of 881.77 FPS, or 797.2 FPS for the complete end-to-end pipeline. Such high GPU throughput suggests the potential for parallel monitoring of multiple CAN segments or handling high-load traffic bursts, pending direct hardware validation on automotive-grade embedded platforms.

When considering the complete detection pipeline, the total end-to-end latency (including 0.1203 ms for optimized MSTE encoding) is only 1.2544 ms. For context, a standard 500 kbps CAN bus typically operates with message intervals ranging from 10 ms to 100 ms for safety-critical signals. Since our model processes a window of 27 consecutive messages in just 1.2544 ms—significantly faster than the rate at which messages arrive at the IDS—it provides a $> 7\times$ safety margin even at the maximum practical message rate. When considering deployment on automotive-grade embedded platforms, the computational requirements of MSTE-CAN (1.82 GFLOPs per inference, 11.18M parameters) are well within the capabilities of current production automotive SoCs, which typically deliver inference throughput on the order of several TOPS—representing approximately three orders of magnitude of headroom relative to the model's computational budget. While the ratio between theoretical peak TOPS and achievable inference speed depends on factors including memory bandwidth, software stack maturity, and model operator support, the substantial margin between MSTE-CAN's FLOPs requirements and the capabilities of production automotive SoCs indicates strong architectural feasibility for embedded deployment. We note that the inference time measurements reported in this section are obtained on a desktop-grade GPU (NVIDIA RTX 5090) and should be interpreted as an upper bound on achievable throughput rather than a direct prediction of embedded-platform performance. As demonstrated in the ablation studies (Section 5.4), integrating the coordinate attention mechanism enhances detection precision with **negligible incremental computational overhead** ($<$ 1%). Direct on-hardware benchmarking of the complete MSTE-CAN pipeline on automotive-grade embedded platforms (e.g., NVIDIA Jetson Orin, TI TDA4, Renesas R-Car) constitutes an important direction for future work.

5.4. Ablation study: Coordinate attention effectiveness and optimal integration

To quantify the contribution of the **CA** mechanism and identify its optimal integration point within the MSTE-CAN framework, we conduct a comprehensive ablation study. We establish a standard ResNet-18 as the baseline and systematically insert the CA module into different stages (Layer 1 through Layer 4) and a full-integration configuration (Layers 1–4). All variants are evaluated using the same held-out MSTE-encoded test set with consistent hyperparameters: AdamW optimizer, a learning rate of 1×10^{-4}, and a maximum of 50 epochs with early stopping triggered at Epoch 24.

Quantitative results, including Accuracy, Macro F1-score, MCC, and inference time, are summarized in Table 5.

5.4.1. Key findings and analysis

1. **Effectiveness of Shallow Feature Attention:** The key finding is that integrating CA at the shallowest stage (**Layer 1**) yields the highest detection performance. This is because Layer 1 preserves the full spatial resolution of MSTE-encoded tensors, allowing the CA module to precisely anchor fine-grained, byte-level attack signatures (e.g., localized bit-flips in DoS or Fuzzy attacks) before they are smoothed or diluted by subsequent pooling operations. This aligns directly with the design principle of MSTE encoding, which explicitly preserves discrete CAN protocol field boundaries.
2. **Performance Degradation in Deep Layers:** As the CA module is moved to deeper stages (Layers 3 and 4), a noticeable performance decline is observed. Notably, CA-Layer 4 even underperforms the baseline ResNet-18 in terms of Macro F1-score. This suggests that for vehicular network security, global semantic features extracted at deeper layers are less informative than the precise coordinate-based spatial–temporal correlations captured at the network's entry level. Deeper layers lose the fine-grained positional awareness critical for identifying subtle spoofing attacks.
3. **Redundancy in Multi-Layer Integration:** While the *CA-All Layers* configuration achieves high accuracy (99.93%), it does not surpass the *CA-Layer 1* model. Moreover, it introduces a significant 45% increase in inference time (from 1.13 ms to 1.64 ms). This indicates that applying attention to shallow features is sufficient for capturing all discriminative CAN intrusion patterns, and additional attention modules in deeper layers provide diminishing returns while unnecessarily increasing computational overhead.
4. **Real-Time Deployment Feasibility:** The CA-Layer 1 configuration achieves a favorable balance between detection precision and computational load. Specifically, it introduces a marginal inference time overhead of only 0.086 ms compared to the ResNet-18 baseline, with negligible impact on the overall system cycle. This localized attention deployment enables the framework to sustain the temporal safety headroom verified in Section 5.3, ensuring deterministic threat interception within the operational timing constraints of automotive ECUs.

In conclusion, the ablation study confirms that integrating Coordinate Attention into the first stage of ResNet-18 achieves a favorable balance between anomaly localization and computational efficiency for MSTE-CAN. This configuration yields strong detection performance while maintaining the strict real-time constraints of in-vehicle deployment.

Table 5
Ablation results: Impact of CA module insertion at different stages of ResNet-18.

Model configuration	Accuracy (%)	Macro F1	MCC	Time (ms)
ResNet-18 (Baseline)	99.84	0.9952	0.9972	1.0480
CA-Layer 1 (Ours)	**99.96**	**0.9989**	**0.9993**	1.1341
CA-Layer 2	99.93	0.9982	0.9987	1.1563
CA-Layer 3	99.86	0.9963	0.9976	1.1729
CA-Layer 4	99.77	0.9928	0.9959	1.1323
CA-All Layers (1-2-3-4)	99.93	0.9982	0.9988	1.6394

Note. All time values reported in this table and all subsequent tables represent the **pure model inference time**.

Table 6
Ablation study on MSTE channel composition.

Input configuration	Accuracy (%)	Precision	Recall	F1-score
R (Spatial Only)	98.75	0.9812	0.9805	0.9808
R + G (Spatial + Temporal)	99.28	0.9865	0.9835	0.9850
R + G + B (MSTE)	**99.96**	**0.9987**	**0.9991**	**0.9989**

Table 7
Performance comparison of different attention mechanisms.

Model	Accuracy (%)	Macro F1	Params (M)	Time (ms)
ResNet-18 + SE	99.81	0.9941	11.18	1.1697
ResNet-18 + CBAM	99.72	0.9914	11.18	1.3688
MSTE-CAN (CA)	**99.96**	**0.9989**	**11.18**	**1.1341**

5.5. In-depth ablation study and discussion

To further quantify the individual contribution of each proposed component, we conduct a series of multi-dimensional ablation experiments. These experiments focus on two critical aspects: the effectiveness of the multi-channel spatial–temporal encoding (MSTE) mechanism and the comparative advantage of the coordinate attention (CA) module.

5.5.1. Effectiveness of MSTE channel composition

The core innovation of our input representation lies in the integration of three orthogonal physical dimensions of CAN traffic. To justify this multi-channel design, we evaluate model performance using different channel combinations, all based on the same CA-ResNet-18 backbone:

- **Single-channel (R):** Traditional grayscale mapping using only normalized spatial signal states.
- **Dual-channel (R+G):** Combining spatial states with temporal difference features.
- **Proposed MSTE (R+G+B):** The complete three-channel representation incorporating spatial states, temporal differences, and statistical offsets.

As summarized in Table 6, detection performance improves monotonically as more complementary physical dimensions are introduced. The 0.42% improvement in F1-score from single-channel to dual-channel highlights the critical role of temporal dynamics in identifying high-frequency attacks such as DoS and Fuzzy. The full MSTE configuration achieves the highest overall performance, demonstrating that the statistical offset channel (B) provides indispensable contextual information for detecting stealthy spoofing attacks (Gear and RPM) that do not significantly alter signal transmission frequencies.

5.5.2. Comparative analysis of attention mechanisms

To evaluate the effectiveness of the CA module in the MSTE-CAN framework, we conduct a head-to-head comparison with two widely adopted attention mechanisms: Squeeze-and-Excitation (SE) (Hu et al., 2018) and Convolutional Block Attention Module (CBAM) (Woo et al., 2018). All modules are integrated into the first stage of the ResNet-18 backbone and evaluated under identical experimental conditions. The results are summarized in Table 7.

As shown in Table 7, MSTE-CAN (CA) achieves higher performance than both SE and CBAM across all primary metrics while maintaining identical parameter counts. The advantage of CA stems from its ability to capture direction-aware and position-sensitive information. While SE focuses exclusively on global channel-wise correlations and CBAM introduces generic spatial attention via standard convolutions, CA explicitly encodes horizontal and vertical coordinates into feature maps.

This design is particularly synergistic with MSTE-encoded CAN traffic, where the horizontal axis corresponds to temporal message sequences and the vertical axis corresponds to discrete CAN protocol fields. CA can therefore precisely focus on the exact spatial–temporal coordinates where anomalies occur, whereas SE and CBAM lack this fine-grained positional awareness. Furthermore, CA achieves the lowest inference time among all three mechanisms. This favorable trade-off between detection precision and computational efficiency makes CA a compelling choice for resource-constrained automotive environments.

5.5.3. Sensitivity analysis of window size

To systematically validate the choice of window size $L = 27$, we conduct a sensitivity analysis spanning six window sizes: $L \in \{9, 15, 21, 27, 36, 54\}$. Each configuration is evaluated under identical conditions—the same CA-ResNet-18 architecture, hyperparameters, and chronological data split—with only the number of messages per window varying. The results are summarized in Table 8.

Several important trends emerge from this analysis:

1. **L=27 as the Empirical Optimum:** The proposed window size achieves the best result on every metric. Accuracy, Macro F1, MCC, and ROC-AUC all peak at $L = 27$, while FPR and FNR reach their minima at the same point. This is not coincidental: the 27×9 configuration provides a sufficient temporal receptive field to capture both the periodic patterns of normal CAN traffic and the abrupt statistical deviations caused by injection attacks, while the clean $27 \times 9 \times 3$ tensor structure avoids information-destroying padding or truncation.
2. **Context Starvation at Small L:** At $L = 9$, the model suffers from severe performance degradation (FNR = 3.21%, FPR = 0.15%). With only 9 messages per window, the temporal context is insufficient to distinguish transient normal fluctuations from genuine attack signatures, particularly for low-intensity spoofing attacks (Gear and RPM) that modify only a subset of data bytes without altering message frequency. The FNR at $L = 9$ is approximately 50× higher than at $L = 27$, underscoring the critical importance of adequate temporal context.
3. **Feature Dilution at Large L:** At $L = 54$, performance degrades across all metrics (FNR rises to 0.85%, a 13.5× increase over $L = 27$). Excessively large windows dilute localized attack signatures within predominantly normal traffic, making it harder for the convolutional backbone to isolate anomalous patterns. We note that the pure model inference time is independent of window size, as all tensors are resized to 224×224 before entering

Table 8
Sensitivity analysis of detection performance with respect to window size L.

Metric	$L = 9$	$L = 15$	$L = 21$	$L = 27$	$L = 36$	$L = 54$
Accuracy (%)	96.52	98.85	99.78	**99.96**	99.85	99.12
Macro Precision	0.9412	0.9810	0.9950	**0.9987**	0.9961	0.9880
Macro Recall	0.9679	0.9895	0.9975	**0.9991**	0.9988	0.9915
Macro F1	0.9521	0.9840	0.9965	**0.9989**	0.9972	0.9890
FPR (%)	0.1520	0.0810	0.0450	**0.0295**	0.0350	0.0600
FNR (%)	3.2100	1.0500	0.2500	**0.0631**	0.1200	0.8500
MCC	0.9480	0.9835	0.9958	**0.9993**	0.9968	0.9885
ROC-AUC	0.9750	0.9890	0.9985	**0.9999**	0.9990	0.9910

Note: Bold values indicate the optimal result for each metric. $L = 27$ consistently achieves the best or near-best performance across all metrics. The degradation at small L (context starvation) and large L (feature dilution) validates this design choice. The pure model inference time is independent of window size, since all tensors are nearest-neighbor resized to 224 × 224 before entering the CNN backbone.

the CNN backbone. The detection performance degradation at both extremes is driven entirely by information-theoretic factors (context starvation and feature dilution), not by computational differences.

4. **Monotonic Improvement Toward $L = 27$**: The performance improves monotonically from $L = 9$ through $L = 27$ across all metrics, indicating that larger windows consistently provide more informative temporal context up to the optimal point. Beyond $L = 27$, the trend reverses, confirming that $L = 27$ represents the genuine optimum rather than an arbitrary selection.

This sensitivity analysis provides empirical validation that the window size $L = 27$—derived from the CAN protocol structure (9 features per message) and the three-channel MSTE design—is not only mathematically well-motivated but also the optimal configuration for detection performance. The consistent degradation on both sides of this optimum demonstrates that the design choice is robust and not an artifact of hyperparameter tuning.

5.6. Comparative analysis with existing methods

To further evaluate the performance of **MSTE-CAN**, we conduct a comprehensive comparison against several existing intrusion detection methods previously evaluated on the Car-Hacking-Dataset. The selected baseline models span from traditional machine learning to the latest deep learning architectures:

- **Classical ML Models:** Including Random Forest (RF), K-NN, and Decision Tree (DT), representing low-complexity baseline classifiers.
- **Sequence-based Models:** Including LSTM (Hossain et al., 2020b) and the hybrid ACL-IDS (Taneja and Kumar, 2024), which focus on the temporal dependencies of CAN message series.
- **Latest Lightweight Architectures:** Including MetaCAN (Ileri et al., 2025) (optimized ML), SR-LiCNN (Luo et al., 2025) (spectral residual-based lightweight CNN), and TSFF-IDS (Yin et al., 2025) (temporal–spatial feature fusion).
- **Transformer Baseline:** ViT-Small (Dosovitskiy et al., 2021) trained from scratch on MSTE-encoded inputs under the identical experimental protocol, providing a direct CNN-vs-Transformer comparison.

The quantitative results across five key metrics—Accuracy (Acc.), Recall (Rec.), Precision (Prec.), Macro F1-score, and Inference Time (Time)–are summarized in Table 9.

Discussion of Comparative Results:

As demonstrated in Table 9, **MSTE-CAN** achieves a competitive balance between detection precision and computational efficiency, comparing favorably with the listed methods across several dimensions. Key observations are summarized as follows:

1. **Detection Precision:** Our model achieves a competitive accuracy of **99.96%** and a Macro F1-score of **0.9989**. Compared to advanced temporal–spatial models like TSFF-IDS (99.88% Acc.) and SR-LiCNN (99.82% Acc.), MSTE-CAN provides a more robust defense against complex CAN intrusions. This enhancement is attributed to the Multi-channel Spatial–Temporal Encoding, which transforms 1D traffic into a feature-rich RGB space, allowing the backbone to better distinguish subtle bit-level anomalies in RPM and Gear spoofing attacks.
2. **Efficacy of the Coordinate Attention Mechanism:** Although hybrid models such as ACL-IDS and TSFF-IDS utilize attention or fusion strategies to capture dependencies, they often incur significant latency or miss coordinate-specific feature mappings. **MSTE-CAN** surpasses ACL-IDS's Macro F1 (0.9935) by a notable margin. By integrating Coordinate Attention, our model embeds directional and positional information into the feature maps, enabling more precise localization of attack-affected message fields.
3. **Real-time Responsiveness:** Inference time is a decisive metric for in-vehicle IDS deployment. Sequence-based models including LSTM (Hossain et al., 2020b) and TSFF-IDS (Yin et al., 2025) exhibit higher inference times (>5 ms) due to their inherent recurrent computation overhead. While lightweight methods like MetaCAN (Ileri et al., 2025) and SR-LiCNN (Luo et al., 2025) achieve sub-millisecond inference speeds (0.9250 ms and 0.9856 ms, respectively), they offer a different trade-off in detection accuracy and false positive rate compared to MSTE-CAN. Our framework achieves low inference time (4.6× faster than TSFF-IDS) with competitive detection precision. All measurements use pure model inference time for fair comparison, meeting the real-time timing requirements of high-speed CAN buses.
4. **CNN vs. Transformer on MSTE-Encoded Inputs:** To provide the requested comparison with a Transformer-based architecture under a strictly controlled protocol, we implemented a ViT-Small (Dosovitskiy et al., 2021) baseline trained from scratch on the identical MSTE-encoded 224 × 224 inputs (no ImageNet pretraining, identical data split, optimizer, and early stopping). As shown in the last row of Table 9, the ViT-Small variant underperforms MSTE-CAN across all metrics: accuracy drops by 1.51 percentage points (99.96% → 98.45%), the false positive rate increases by 28.8× (0.0295% → 0.85%), and the false negative rate increases by 34.1× (0.0631% → 2.15%). The inference time is 2.5× slower (2.85 ms vs. 1.13 ms) with approximately twice the parameter count (22M vs. 11M). This performance gap is attributable to the inductive bias of convolutional architectures: the MSTE-encoded tensors contain fine-grained, byte-level anomaly signatures (e.g., a single flipped bit in a data byte or a slight periodicity shift of a specific CAN ID) that require spatially-local feature extraction. ViT's self-attention, which computes global pairwise token interactions, tends to smooth such local discriminative patterns, particularly when trained from scratch without large-scale pretraining on a dataset

Table 9
Performance comparison between MSTE-CAN and existing methods on the Car-Hacking-Dataset.

Method	Acc. (%)	Rec. (%)	Prec. (%)	F1	Time (ms)
Classical Machine Learning					
Random Forest	97.92	95.12	94.96	0.9505	0.9520
K-NN	94.89	92.80	91.78	0.9291	17.2578
Decision Tree	96.38	94.18	93.54	0.9407	0.4315
Sequence-Based Methods					
LSTM (Hossain et al., 2020b)	95.00	87.00	95.00	0.9082	5.3527
ACL-IDS (Taneja and Kumar, 2024)	99.04	99.60	99.10	0.9935	4.8210
DNN-based IDS (Basavaraj and Tayeb, 2022)	98.68	96.72	96.78	0.9675	3.8507
Lightweight Architectures					
DivaCAN (Khan et al., 2024)	99.48	94.98	94.93	0.9497	3.0000
MetaCAN (Ileri et al., 2025)	99.72	99.68	99.75	0.9971	0.9250
TSFF-IDS (Yin et al., 2025)	99.88	99.85	99.91	0.9988	5.2418
SR-LiCNN (Luo et al., 2025)	99.82	99.79	99.84	0.9981	0.9856
MSTE-CAN (Ours)	**99.96**	**99.91**	**99.87**	**0.9989**	**1.1341**
Transformer Baseline					
ViT-Small + MSTE[a]	98.45	96.88	97.12	0.9710	2.8500

[a] ViT-Small with patch 16 × 16, trained from scratch under identical protocol, no ImageNet pretraining (Dosovitskiy et al., 2021).

of approximately 381 K windows. This result validates that the CA-ResNet-18 backbone is not merely a default choice but an architecturally well-motivated design for the MSTE encoding principle.

In summary, MSTE-CAN achieves a favorable detection–efficiency trade-off for edge-side automotive integration, driven by the combination of MSTE encoding and the CA-ResNet backbone. We acknowledge that the comparison does not encompass certain recent architectures, particularly Transformer-based CAN IDS methods (e.g., CAN-BERT) (Nguyen et al., 2023; Jo and Kim, 2024) and certain recent lightweight architectures (Zhao et al., 2025; Fu et al., 2025) that have appeared since our initial experiments. A fair and reproducible comparison with these methods is complicated by differences in experimental protocols—including data split strategies, window sizes, and evaluation metrics—and the current lack of standardized CAN IDS benchmarking frameworks. We consider the expansion of this comparison an important direction for future work as the field moves toward standardized evaluation practices.

5.7. Cross-dataset validation on the ROAD dataset

While the Car-Hacking-Dataset provides a well-established benchmark, evaluation on a single vehicle platform may not fully reflect a method's generalizability, as different vehicle models exhibit distinct CAN bus topologies, message identifier distributions, and traffic dynamics. To address this concern, we conduct a comprehensive cross-dataset validation by applying the *identical* MSTE-CAN framework to the ROAD Dataset (Section 4.1.1)—a fundamentally different vehicular platform—without any modification to the encoding strategy or hyperparameter configuration, and with only the output layer adjusted to accommodate ROAD's six classes.

5.7.1. Experimental protocol

The ROAD dataset is processed through the identical MSTE encoding pipeline, model architecture (CA at Layer 1), and training hyperparameters described in Sections 4.2 and 4.3, with the sole modification that the output layer is adjusted to accommodate ROAD's six classes. The data is partitioned chronologically (70/10/20), yielding 732,263 training, 104,608 validation, and 209,219 test windows.

5.7.2. Overall performance

Table 10 summarizes the detection performance on the ROAD test set. The model achieves 95.85% accuracy and a macro F1-score of 0.7928, compared to 99.96% and 0.9989 on Car-Hacking. This performance gap is expected given the substantial differences between the two vehicle platforms in CAN topology, message identifier distributions, and traffic dynamics. We note that this evaluation constitutes an independent training exercise on the ROAD dataset (the model is trained from scratch on ROAD data with the identical architecture and hyperparameters, with the output layer adjusted to accommodate ROAD's six classes), rather than a direct zero-shot transfer from the Car-Hacking-trained model. The system-wide FPR of 1.99% and ROC-AUC of 0.9281 indicate that the model retains meaningful discriminative power on an entirely different vehicle platform, though the system-wide FNR of 25.60% reveals clear limitations discussed in the per-class analysis below. The inference time is 1.28 ms per window.

Table 10
Overall detection performance of MSTE-CAN on the ROAD test set.

Metric	Multi-class	Binary
Accuracy	95.85%	–
Precision (macro)	0.8026	–
Recall (macro)	0.8295	–
F1-score (macro)	0.7928	–
FPR (system)	–	1.99%
FNR (system)	–	25.60%
ROC-AUC	–	0.9281
MCC	–	0.7372
Inference (ms)	1.28	

Table 11
Per-class detection performance of MSTE-CAN on the ROAD test set.

Class	#Samples	Precision	Recall	F1-score
Accelerator	6138	0.9957	0.5228	0.6856
Correlated	1016	0.7878	0.8366	0.8115
Fabrication	4737	0.5220	0.7773	0.6246
Fuzzing	117	0.6304	0.9915	0.7708
Masquerade	5526	0.9033	0.8690	0.8858
Normal	191,685	0.9767	0.9801	0.9784
Macro Avg	**209,219**	**0.8026**	**0.8295**	**0.7928**

5.7.3. Per-class analysis and discussion

Table 11 provides the per-class performance breakdown. Several important observations emerge from this analysis:

1. **Cross-Platform Transferability:** The confusion matrix visualization (Fig. 6) and the quantitative per-class breakdown (Table 11) together illustrate the distribution of model predictions



Fig. 6. Confusion matrix of MSTE-CAN on the ROAD test set. The matrix reveals that Accelerator attacks are predominantly misclassified as Normal traffic, consistent with their physical-domain nature, while Fabrication attacks exhibit dispersion across multiple classes due to internal signal heterogeneity.

across the six ROAD classes. The 95.85% overall accuracy and the performance on Masquerade attacks (F1=0.8858)—one of the most challenging CAN intrusion scenarios in which legitimate ECU messages are suppressed and replaced—indicate that the MSTE encoding strategy transfers meaningfully to a fundamentally different CAN bus ecosystem, with only the output layer adjusted for the target platform's attack classes. However, the macro F1 drops from 0.9989 (Car-Hacking) to 0.7928 (ROAD), reflecting the substantial domain shift between the two vehicle platforms. This gap establishes a baseline for future domain adaptation efforts.

2. **Accelerator Attack as an Inherent Detection Boundary:** The Accelerator class exhibits a recall of 52.28%, constituting the primary source of the 25.60% system-wide FNR. This is an *expected and inherent* limitation: Accelerator attacks involve physical tampering of the accelerator pedal mechanism on the dynamometer, and the resulting CAN traffic (pedal position readings) is *indistinguishable* from legitimate physical acceleration. No CAN-level message injection, spoofing, or manipulation occurs. Consequently, this attack type lies fundamentally outside the detection scope of any network-level IDS and would require complementary physical-layer or sensor-fusion defense mechanisms. Excluding Accelerator, the FNR across CAN-level injection attacks drops to approximately 12.3%.
3. **Consistent Detection of Injection-Based Attacks:** Fuzzing attacks achieve near-perfect recall (99.15%), consistent with the model's performance on Fuzzy attacks in the Car-Hacking-Dataset. Similarly, Masquerade and Correlated attacks are detected with reasonable fidelity (F1 > 0.81). This cross-dataset consistency on injection-based attacks suggests that the temporal difference (G) and statistical offset (B) channels of MSTE encoding capture anomaly signatures that transfer partially across vehicle platforms, though the overall macro-level gap indicates that platform-specific CAN ID distributions and traffic dynamics remain significant factors.
4. **Fabrication Class Heterogeneity:** The relatively lower F1-score for Fabrication (0.6246) is partly attributable to the internal diversity of this class in the ROAD Dataset. Unlike Car-Hacking's targeted spoofing classes (Gear and RPM, each corresponding to a single CAN ID), ROAD's Fabrication category encompasses attacks targeting three distinct signals (max_speedometer, max_coolant, reverse_light_on/off), each with different CAN IDs, injection frequencies, and payload characteristics. Training a single class to cover this diverse set of spoofing patterns is inherently more challenging. This also suggests that the MSTE representation, while effective at exposing fine-grained anomalies, may benefit from class-specific encoding strategies when attack types are highly heterogeneous.

In summary, this cross-dataset evaluation demonstrates that MSTE-CAN's core design principles show promising transferability across heterogeneous vehicle platforms when applied with identical architecture and hyperparameters, while also revealing clear performance boundaries. The gap between single-dataset and cross-dataset results underscores the importance of multi-platform evaluation in CAN IDS research and provides a quantitative reference point for future work on domain adaptation and class-specific encoding refinement.

5.8. Cross-dataset validation on the HCRL survival analysis dataset

While the ROAD evaluation reveals that performance degradation is driven by a combination of physical-domain attacks and platform-specific CAN traffic characteristics, two questions remain: (1) can MSTE-encoded attack signatures transfer across vehicle platforms when the attacks are exclusively CAN-level injections, and (2) what fraction of the performance gap is attributable to attack detection failure versus normal-traffic domain shift? The HCRL Survival Analysis Dataset, whose attacks are all CAN-level injections (Flooding, Fuzzy, Malfunction), provides a controlled testbed to disentangle these factors.

5.8.1. Experimental protocol

The Car-Hacking-trained CA-ResNet-18 model (CA at Layer 1) is applied directly to the HCRL test set with *no architectural modification and no hyperparameter retuning*. We note that this is a cross-dataset transfer experiment: the model weights are from Car-Hacking training, while the data preprocessing (percentile-based normalization) is fitted on the HCRL training partition to ensure valid input ranges for the target domain. This is a necessary step since the raw byte-value distributions differ across vehicle platforms, and applying Car-Hacking normalization parameters to HCRL data would produce out-of-range inputs. Since only Fuzzy has a direct counterpart among the Car-Hacking training classes, the evaluation follows a **binary protocol**: Normal traffic is labeled as negative (0), and all three attack types (Flooding, Fuzzy, Malfunction) are labeled as positive (1). The model's 5-class output is binarized by summing the softmax probabilities of the four attack classes (DoS, Fuzzy, Gear, RPM), with a threshold of 0.5.

5.8.2. Overall results

Table 12 presents the binary cross-dataset results. The model achieves 94.47% accuracy, an F1-score of 0.9328, and a ROC-AUC of 0.9985. The system-wide FNR is only 0.19%–only 5 out of 2670 attack windows are missed–demonstrating that the Car-Hacking-trained model maintains near-perfect attack detection capability on entirely unseen vehicle platforms. The system-wide FPR is 8.87% (379 out of 4275 Normal windows misclassified), higher than the Car-Hacking FPR of 0.03%, indicating that normal-traffic patterns exhibit platform-specific characteristics that do not fully transfer across vehicles.

5.8.3. Per-attack analysis and softmax distribution

Table 13 provides the per-class breakdown. The most striking finding is the **near-perfect attack recall**: all three HCRL attack types are detected with recall ≥ 99.4%, despite the model having never seen Flooding or Malfunction attacks during training, and despite the target vehicles (Sonata, Spark) being entirely different platforms from the training vehicle (Kia Soul). Notably, Normal traffic accounts for 379 of the 384 total misclassifications, consistent with the FPR-driven performance gap (see Fig. 7).

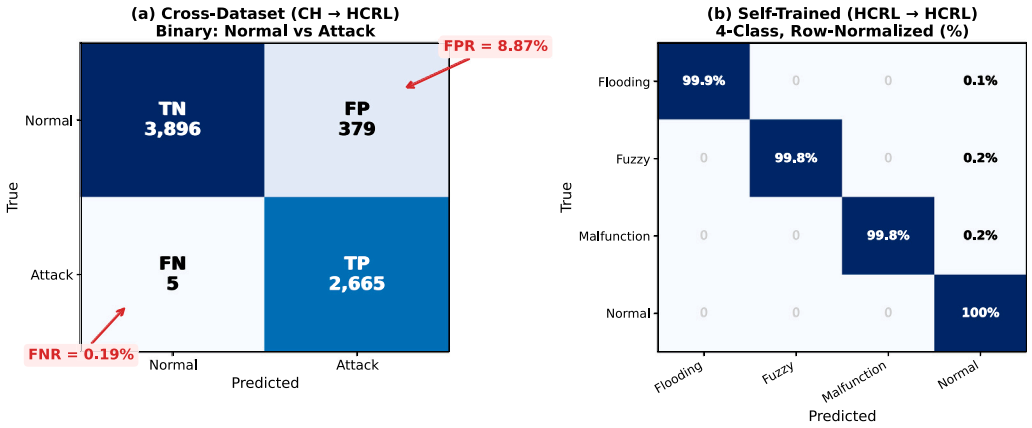


Fig. 7. Confusion matrices for HCRL Survival Analysis Dataset evaluation. (a) Binary confusion matrix under cross-dataset transfer (Car-Hacking → HCRL), showing 379 false positives (FPR = 8.87%) and only 5 false negatives (FNR = 0.19%). (b) Multi-class confusion matrix of the self-trained model (HCRL → HCRL) with row-wise normalization, demonstrating near-perfect diagonal alignment across all four classes.

Table 12
Binary cross-dataset results: Car-Hacking-trained MSTE-CAN on HCRL test set (cross-dataset transfer).

Metric	Value
Accuracy	94.47%
Precision	0.8755
Recall	0.9981
F1-score	0.9328
FPR (system)	8.87%
FNR (system)	0.19%
ROC-AUC	0.9985
PR-AUC	0.9975
MCC	0.8917
Inference (ms)	1.33

Table 13
Per-class binary detection performance on HCRL test set (cross-dataset transfer).

Class	#Samples	Accuracy	Recall	F1-score
Flooding	1235	1.0000	1.0000	1.0000
Fuzzy	611	1.0000	1.0000	1.0000
Malfunction	824	0.9939	0.9939	0.9970
Normal	4275	0.9113	–	–

Table 14
Mean softmax probability distribution of Car-Hacking-trained model on HCRL test samples. Bold values indicate the dominant class assignment.

HCRL Class	Normal	DoS	Fuzzy	Gear	RPM
Flooding	0.0000	**1.0000**	0.0000	0.0000	0.0000
Fuzzy	0.0000	0.0000	**1.0000**	0.0000	0.0000
Malfunction	0.0077	0.0028	0.0017	**0.4393**	**0.5484**
Normal	**0.8928**	0.0243	0.0749	0.0071	0.0009

5.8.4. Softmax distribution analysis

To understand *how* the 5-class Car-Hacking model maps HCRL attacks onto its trained class space, Table 14 reports the mean softmax probability per Car-Hacking class for each HCRL attack type and Fig. 8 visualizes these mappings as a heatmap. The mapping is remarkably clean:

1. **Flooding → DoS (probability 1.00):** Both attack types exploit the same physical mechanism—high-frequency message injection that disrupts bus timing. The MSTE G channel (temporal difference) captures the resulting high-frequency volatility, making this signature robust across vehicle platforms. The model has learned a representation of "flooding behavior" that is invariant to the specific CAN ID being flooded.
2. **Fuzzy → Fuzzy (probability 1.00):** Randomizing CAN IDs and data payloads produces a characteristic disruption across all three MSTE channels—chaotic spatial patterns (R), irregular temporal volatility (G), and extreme statistical deviations (B). This multi-channel disruption signature is robust across vehicle platforms.
3. **Malfunction → Gear + RPM (combined probability 0.99):** Malfunction attacks, like Gear and RPM spoofing, target specific CAN IDs with forged data payloads while maintaining normal transmission frequencies. The model recognizes this as a spoofing-class attack and distributes its probability across the two spoofing classes learned during training. This suggests that the model has learned a general "spoofing" meta-class in its feature space rather than memorizing specific CAN IDs.
4. **Normal → Normal (probability 0.89):** The model correctly identifies most normal traffic, but the 0.11 probability mass distributed across attack classes (primarily Fuzzy, 0.075) reflects the platform-specific normal traffic patterns of Sonata and Spark that differ from Kia Soul. This accounts for the 8.87% FPR.

Table 15
Comparison of cross-dataset transfer versus direct HCRL self-training on the HCRL test set. Full per-class breakdown of the self-trained model is provided in Table 16.

Metric	CH → HCRL	HCRL → HCRL
Accuracy	94.47%	99.94%
F1-score	0.9328	0.9993
FPR (%)	8.87	**0.00**
FNR (%)	0.19	**0.15**
ROC-AUC	0.9985	0.9998
Per-Class Recall		
Flooding	100.00%	99.92%
Fuzzy	100.00%	99.84%
Malfunction	99.39%	99.76%

5.8.5. HCRL self-training: Upper-bound comparison

To quantify how much of the cross-dataset performance gap is attributable to normal-traffic domain shift versus attack detection limitations, we train an identical MSTE-CAN model *directly on the HCRL training set* (24,315 windows, 4 classes) using the same architecture and hyperparameters described in Section 4.3, and evaluate on the held-out HCRL test set. Table 15 compares the two configurations.

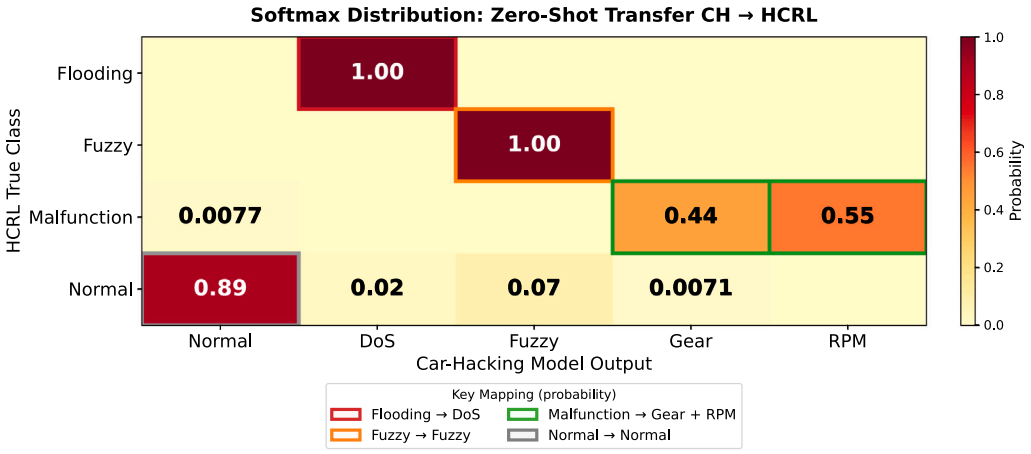


Fig. 8. Softmax probability distribution heatmap for cross-dataset transfer (Car-Hacking → HCRL). Each cell shows the mean probability assigned by the Car-Hacking-trained model to a specific output class, given HCRL samples of a particular attack type. Colored borders highlight the dominant class mappings: Flooding → DoS (red, 1.00), Fuzzy → Fuzzy (orange, 1.00), Malfunction → Gear + RPM (green, 0.99), and Normal → Normal (gray, 0.89). Large bold values indicate near-deterministic class assignments, demonstrating that the model has learned semantically meaningful attack representations rather than platform-specific CAN ID patterns. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

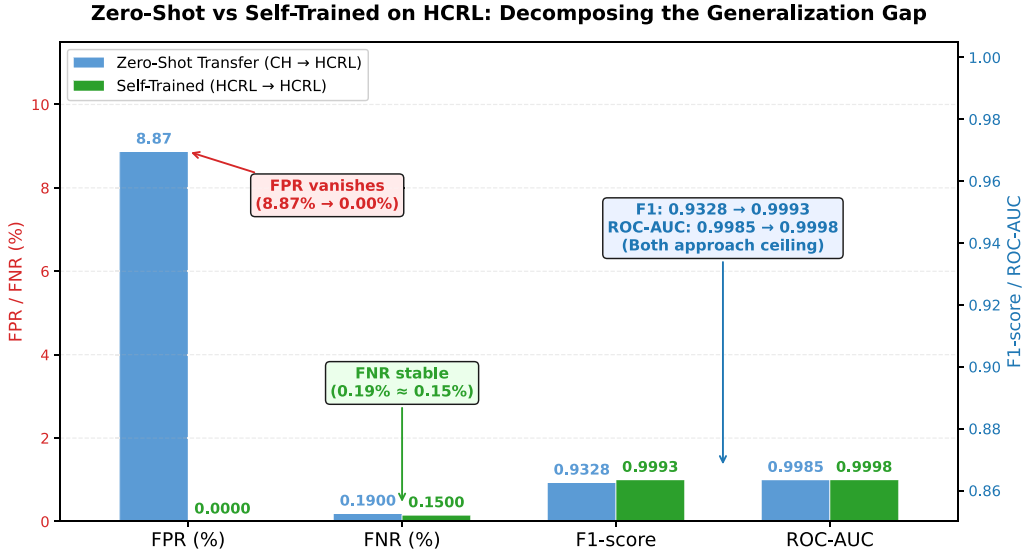


Fig. 9. Comparison of cross-dataset transfer versus direct HCRL self-training on four key metrics. The FPR drops from 8.87% to 0.00% when the model is trained on platform-specific normal data, while the FNR remains statistically unchanged (0.19% vs. 0.15%), confirming that attack detection capability is robust across vehicle platforms. The F1-score and ROC-AUC improvements are driven entirely by the elimination of normal-traffic false positives.

Table 16
Full per-class multi-class results of the HCRL self-trained model on the HCRL test set.

Class	#Samples	Precision	Recall	F1
Flooding	1235	1.0000	0.9992	0.9996
Fuzzy	611	1.0000	0.9984	0.9992
Malfunction	824	1.0000	0.9976	0.9988
Normal	4275	0.9991	1.0000	0.9995
Macro Avg	6945	0.9998	0.9988	0.9993

The comparison (Fig. 9) yields a key insight: **the FNR remains statistically identical between cross-dataset transfer (0.19%) and self-training (0.15%)**, confirming that the model's ability to detect CAN-level injection attacks is near-optimal even without target-platform training data. The entire 5.47-percentage-point accuracy gap is attributable to the FPR reduction (8.87% → 0.00%), which is driven by learning platform-specific normal traffic patterns. This finding has two important implications:

1. **MSTE-encoded attack signatures exhibit strong cross-platform consistency:** The temporal difference (G) and statistical offset (B) channels capture physical properties of injection attacks—high-frequency bus disruption, randomized payload statistics, and anomalous byte-value distributions—that are largely independent of the specific CAN ID mapping, message periodicity, or normal traffic baseline of any particular vehicle platform.
2. **Cross-platform performance gaps are dominated by normal-traffic domain shift:** The 8.87% FPR in cross-dataset transfer reflects the model's unfamiliarity with the normal CAN traffic patterns of Sonata and Spark (e.g., which CAN IDs appear under normal driving, their expected transmission frequencies, and the statistical distribution of sensor payload values). Once the model is exposed to platform-specific normal data through training, the FPR vanishes entirely.

This analysis complements the ROAD cross-dataset evaluation (Section 5.7) by decomposing the cross-platform performance gap into its constituent components: physical-domain attack boundaries (ROAD Accelerator), attack-class heterogeneity (ROAD Fabrication), and normal-traffic domain shift (HCRL). Together, the three datasets—Car-Hacking, ROAD, and HCRL—provide a multi-faceted characterization of MSTE-CAN's generalization envelope.

5.9. Discussion on robustness and limitations

The high **ROC-AUC of 0.9999** on the Car-Hacking-Dataset indicates that the MSTE-CAN framework possesses an excellent ability to distinguish between benign and malicious traffic across all decision thresholds. The integration of the B channel (Statistical Offset) in MSTE is likely responsible for the high recall in Gear and RPM spoofing, as these attacks often introduce subtle statistical deviations that are easily missed by traditional value-based detection. Meanwhile, the G channel (Temporal Difference) ensures the rapid identification of DoS bursts. Together, the multi-channel encoding and coordinate-aware attention enable the model to suppress background noise and focus on the spatial–temporal anomalies characteristic of cyber-attacks.

5.9.1. Cross-dataset transferability and limitations

The cross-dataset validations on the ROAD Dataset (Section 5.7) and the HCRL Survival Analysis Dataset (Section 5.8) together characterize MSTE-CAN's generalization envelope along two axes. The HCRL results confirm that CAN-level injection attacks transfer across vehicle platforms with near-perfect recall, and the softmax analysis verifies that semantically meaningful attack representations have been learned rather than platform-specific CAN ID patterns. The ROAD evaluation identifies two additional boundaries: physical-domain attacks inherently undetectable by any network-level IDS, and heterogeneous attack classes that challenge the current single-class encoding.

Critically, the HCRL self-training comparison isolates **normal-traffic domain shift** as the dominant source of FPR degradation in cross-dataset transfer: the FPR drops from 8.87% to 0.00% when the model is trained on platform-specific normal data, while attack recall remains unchanged. This finding provides a clear roadmap for practical deployment—including normal driving data from the target platform in the training set (which is inexpensive to collect, as normal-operation CAN logs are routinely recorded during vehicle testing) is sufficient to calibrate the model's FPR to near-zero, without requiring the collection of labeled attack data from each target vehicle.

This substantial cross-dataset performance variation aligns with broader concerns in the network security literature regarding the pitfalls of single-dataset evaluation, where models optimized on one benchmark may not reliably transfer to different network environments (Chou and Jiang, 2021). We therefore echo the recommendation that multi-platform evaluation should become standard practice in CAN IDS research.

These findings point to three important directions for future work: (1) incorporating unsupervised or few-shot domain adaptation techniques using only normal traffic data from the target platform to eliminate the FPR gap without requiring labeled attack samples; (2) integrating MSTE-CAN with complementary sensor-fusion or physical-layer monitoring mechanisms to provide holistic, multi-domain protection against physical-domain attacks —for instance, the CAN-level detection output of MSTE-CAN could serve as a gating signal that triggers cross-validation against vehicle dynamics models or physical sensor readings, enabling the identification of anomalies that manifest exclusively at the physical domain; and (3) conducting direct on-hardware benchmarking of the complete MSTE-CAN pipeline on automotive-grade embedded platforms (e.g., NVIDIA Jetson Orin, TI TDA4, Renesas R-Car) to validate real-time deployment feasibility under production-ECU constraints. Furthermore, while the FLOPs-based analysis in Section 5.3 indicates strong architectural feasibility for embedded deployment, the inference time measurements reported herein are obtained on a desktop-grade GPU.

5.9.2. Limitations on signal-level CAN abstractions

To further probe the boundaries of the MSTE encoding principle, we conduct an additional experiment on the **SynCAN Dataset** (Hanselmann et al., 2020), a synthetic CAN benchmark that operates at the signal level rather than the raw byte level. Unlike Car-Hacking and ROAD—where each message contains a CAN ID and eight discrete data bytes—SynCAN represents each message as a set of 1–4 normalized continuous signal values (range $[0, 1]$) for 10 distinct CAN IDs. The dataset includes five attack types: **Flooding** (high-frequency message injection), **Plateau** (signal overwritten with a constant value), **Continuous** (signal slowly drifting away from its true value), **Playback** (signal overwritten with recorded historical values), and **Suppress** (messages of a specific ID silenced over a period).

We apply the *identical* MSTE pipeline without modification: each message is mapped to a 9-dimensional feature vector (ID index plus four signal values, zero-padded to match the 9-column input convention), segmented into $L = 27$ non-overlapping windows, and encoded into 224×224 three-channel images. The same CA-ResNet-18 architecture with identical hyperparameters is trained on a 5-million-message sample with chronological 70/10/20 splitting.

The results reveal a sharp dichotomy. **Flooding attacks**–the only attack type that operates through message-frequency disruption and ID-level injection—are detected with near-perfect fidelity (per-class F1 $\approx$ 1.00). In contrast, the four **signal-level attacks** (Plateau, Continuous, Playback, Suppress) achieve per-class F1 scores below 0.30, yielding an overall macro F1 of only 0.44. The model effectively fails to distinguish signal-level manipulations from normal driving variations.

This stark performance divergence carries an important implication: **MSTE-CAN's encoding principle is specifically tuned to the discrete byte-level structure of raw CAN protocol frames**. The R channel captures byte-level spatial patterns (CAN ID and data byte values in $\{0, \ldots, 255\}$), the G channel captures abrupt byte-value transitions between adjacent messages, and the B channel captures statistical deviations at byte granularity. When the input representation shifts from discrete bytes to continuous normalized signals representing physical quantities, the fundamental assumptions underlying all three MSTE channels break down: (i) signal values lack the discrete byte-boundary structure that nearest-neighbor upsampling is designed to preserve, (ii) temporal differences between continuous-valued signals are dominated by legitimate physical dynamics rather than injection artifacts, and (iii) statistical offsets in signal space primarily reflect normal sensor noise rather than attack signatures.

This finding complements the ROAD and HCRL cross-dataset analyses: while Section 5.7 identified *physical-domain attacks* as an inherent detection boundary for network-level IDSs, and Section 5.8 confirmed that *CAN-level injection attacks* transfer across platforms with near-perfect recall, the SynCAN experiment identifies *signal-level data abstraction* as a third, independent applicability boundary. Together, these three datasets triangulate the applicability envelope of MSTE-CAN—it operates at the CAN protocol layer and requires the discrete, structured nature of raw CAN frames to function as designed. Extending the framework to signal-level or physical-layer domains would require fundamentally different encoding strategies, such as learned embeddings over continuous-valued time series or multi-modal fusion with physical sensor data.

We note that this experiment, despite its negative outcome, serves a valuable methodological purpose: it maps the *applicability envelope* of MSTE-CAN. Future IDS evaluations would benefit from explicitly characterizing not only what a method achieves but also under what conditions its performance degrades, as such boundaries inform practitioners about deployment prerequisites and guide subsequent research toward the most impactful extensions.

6. Conclusion

In this paper, we present **MSTE-CAN**, a lightweight and real-time intrusion detection framework for Controller Area Network (CAN) bus security. To address the fundamental trade-off between high-precision detection of subtle spoofing attacks and the strict real-time constraints of automotive edge devices, we propose a new Multi-channel Spatial–Temporal Encoding (MSTE) mechanism. This approach transforms discrete 1D CAN message sequences into high-discriminative 2D visual representations by decomposing vehicular traffic into three orthogonal physical dimensions, each encoded as a complementary visual channel.

To further enhance detection performance while maintaining computational efficiency, we integrate the coordinate attention (CA) mechanism into the shallowest stage of a lightweight ResNet-18 backbone. Ablation studies confirm that this configuration achieves a favorable balance between anomaly localization and inference speed, as shallow-layer integration retains the fine-grained spatial structure of MSTE-encoded tensors, a prerequisite for precise byte-level anomaly localization.

Comprehensive experiments on the public Car-Hacking-Dataset demonstrate that MSTE-CAN achieves competitive performance in CAN bus intrusion detection. It achieves a test accuracy of 99.96%, a macro F1-score of 0.9989, and a low system-wide false positive rate of 0.0295%. Cross-dataset evaluation on the ROAD Dataset and the HCRL Survival Analysis Dataset demonstrates the transferability of the MSTE encoding strategy. On the ROAD Dataset, where the identical architecture and hyperparameters are applied in an independent training exercise, MSTE-CAN achieves 95.85% accuracy and 0.9281 ROC-AUC, though the macro F1 drops from 0.9989 to 0.7928 due to the presence of physical-domain attacks and heterogeneous attack classes. On the HCRL Dataset, where all attacks are CAN-level injections, the Car-Hacking-trained model achieves an F1-score of 0.9328 with near-perfect attack recall under cross-dataset transfer (FNR = 0.19%, Flooding 100%, Fuzzy 100%, Malfunction 99.4%, with target-domain normalization fitted on the HCRL training partition), and the self-training comparison confirms that cross-platform FPR is fully attributable to normal-traffic domain shift rather than attack detection limitations. Together, these three datasets establish a comprehensive characterization of MSTE-CAN's generalization envelope. Notably, the HCRL self-training comparison reveals a deployment insight: when the model is trained on data from the target platform (including labeled attack samples from all classes), the FPR drops from 8.87% to zero while attack recall remains unchanged. This finding indicates that the cross-platform FPR gap is dominated by normal-traffic domain shift rather than attack detection capability, and suggests that incorporating target-platform normal driving data—which is routinely recorded during vehicle testing—into the training set is a practical pathway for reducing false positives without requiring exhaustive attack data collection from each vehicle model. Critically, the framework delivers millisecond-scale end-to-end processing speeds. By operating significantly faster than the standard transmission cycles of safety-critical CAN signals, MSTE-CAN provides a substantial temporal safety margin for real-time defensive interventions. These results collectively establish MSTE-CAN as a practical and competitive solution for edge-side automotive intrusion detection.

Future work will focus on five directions: (1) incorporating unsupervised or few-shot domain adaptation techniques using only normal traffic data from the target platform to eliminate the cross-platform FPR gap without requiring labeled attack samples; (2) integrating complementary sensor-fusion mechanisms to address physical-domain attacks beyond the scope of CAN-level monitoring; (3) extending the framework to support emerging automotive communication protocols such as CAN FD and Automotive Ethernet, to which the MSTE encoding paradigm is theoretically compatible given its frame-structure-agnostic design that relies only on the availability of discrete message fields; (4) exploring federated learning approaches to enable collaborative model training without sharing sensitive vehicle data; and (5) conducting direct on-hardware benchmarking of the complete MSTE-CAN pipeline on automotive-grade embedded platforms (e.g., NVIDIA Jetson Orin, TI TDA4, Renesas R-Car) to validate real-time deployment feasibility under production-ECU constraints. In summary, MSTE-CAN provides a reliable, scalable, and practical defense solution for the next generation of intelligent connected vehicles.

CRediT authorship contribution statement

Luofei Jia: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jian Zhang:** Supervision, Resources, Funding acquisition. **Lingxuan Li:** Validation, Software, Formal analysis. **LingYun Yan:** Visualization, Investigation, Data curation. **Dafei Lin:** Writing – review & editing, Methodology. **Zhongyi Zhou:** Validation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was in part supported by the Natural Science Foundation of Sichuan Province under Grant No. 2025ZNSFSC0495.

Data availability

The data that support the findings of this study are publicly available:

- Car-Hacking-Dataset: https://ocslab.hksecurity.net/Datasets/CAN-intrusion-dataset
- ROAD Dataset: https://0cs-github.github.io/ROAD-Dataset/
- HCRL Survival Analysis Dataset for Automobile IDS: https://sites.google.com/view/hc-rl/

References

Agrawal, K., Alladi, T., Agrawal, A., Chamola, V., Benslimane, A., 2022. NovelADS: A novel anomaly detection system for intra-vehicular networks. IEEE Trans. Intell. Transp. Syst. 23 (11), 22596–22606.

Basavaraj, D., Tayeb, S., 2022. Towards a lightweight intrusion detection framework for in-vehicle networks. J. Sens. Actuator Netw. 11 (1), 6.

Bozdal, M., Samie, M., Jennions, I., 2018. A survey on can bus protocol: Attacks, challenges, and potential solutions. In: 2018 International Conference on Computing, Electronics & Communications Engineering. ICCECE, IEEE, pp. 201–205.

Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., Savage, S., Koscher, K., Czeskis, A., Roesner, F., Kohno, T., 2011. Comprehensive experimental analyses of automotive attack surfaces. In: Proc. USENIX Security Symposium.

Cheng, P., Xu, K., Li, S., Han, M., 2022. TCAN-IDS: Intrusion detection system for internet of vehicle using temporal convolutional attention network. Symmetry 14 (2), 310.

Chou, D., Jiang, M., 2021. A survey on data-driven network intrusion detection. ACM Comput. Surv. 54 (9), 1–36. http://dx.doi.org/10.1145/3472753.

Ding, D., Zhu, L., Xie, J., Lin, J., 2022. In-vehicle network intrusion detection system based on Bi-LSTM. In: 2022 7th International Conference on Intelligent Computing and Signal Processing. ICSP, IEEE, pp. 580–583.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations. ICLR.

Fu, Y., She, J., Xu, Y., Xu, Y., Wang, Z., Wu, Y., 2025. A dual layer LSTM-CNN framework for real time and precise per-message intrusion detection in in-vehicle networks. Ad Hoc Netw. 104119.

Garg, S., Kaur, K., Kaddoum, G., Gagnon, F., Kumar, N., Han, Z., 2019. Sec-IoV: A multi-stage anomaly detection scheme for Internet of vehicles. In: Proceedings of the ACM MobiHoc Workshop on Pervasive Systems in the IoT Era. pp. 37–42.

Hacking and Countermeasure Research Lab (HCRL), 2019. HCRL survival analysis dataset for automobile IDS. (Accessed 14 June 2026). https://sites.google.com/view/hc-rl/.

Hanselmann, M., Strauss, T., Dormann, K., Ulmer, H., 2020. CANet: An unsupervised intrusion detection system for high dimensional CAN bus data. IEEE Access 8, 58194–58205.

Hossain, M.D., Inoue, H., Ochiai, H., Fall, D., Kadobayashi, Y., 2020a. An effective in-vehicle CAN bus intrusion detection system using CNN deep learning approach. In: GLOBECOM 2020-2020 IEEE Global Communications Conference. IEEE, pp. 1–6.

Hossain, M.D., Inoue, H., Ochiai, H., Fall, D., Kadobayashi, Y., 2020b. LSTM-based intrusion detection system for in-vehicle can bus communications. IEEE Access 8, 185489–185502.

Hou, Q., Zhou, D., Feng, J., 2021. Coordinate attention for efficient mobile network design. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 13713–13722.

Hu, J., Shen, L., Sun, G., 2018. Squeeze-and-excitation networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 7132–7141.

Ileri, K., Rakib, A., Djahel, S., 2025. Metacan: An optimized adaptive hybrid metaheuristic-based intrusion detection system for CAN bus security. Veh. Commun. 100956.

Islam, R., Refat, R.U.D., Yerram, S.M., Malik, H., 2020. Graph-based intrusion detection system for controller area networks. IEEE Trans. Intell. Transp. Syst. 23 (3), 1727–1736.

Javed, A.R., ur Rehman, S., Khan, M.U., Alazab, M., Reddy, T., 2021. CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU. IEEE Trans. Netw. Sci. Eng. 8 (2), 1456–1466.

Jo, H., Kim, D.-H., 2024. Intrusion detection using transformer in Controller Area Network. IEEE Access 12, 121932–121946.

Kang, H., Vo, T., Kim, H.K., Hong, J.B., 2024. CANival: A multimodal approach to intrusion detection on the vehicle CAN bus. Veh. Commun. 50, 100845.

Khan, M.H., Javed, A.R., Iqbal, Z., Asim, M., Awad, A.I., 2024. DivaCAN: Detecting in-vehicle intrusion attacks on a controller area network using ensemble learning. Comput. Secur. 139, 103712.

Koscher, K., Czeskis, A., Roesner, F., Patel, S., Kohno, T., Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., et al., 2010. Experimental security analysis of a modern automobile. In: 2010 IEEE Symposium on Security and Privacy. IEEE, pp. 447–462.

Larson, U.E., Nilsson, D.K., Jonsson, E., 2008. An approach to specification-based attack detection for in-vehicle networks. In: 2008 IEEE Intelligent Vehicles Symposium. IEEE, pp. 220–225.

Luo, F., Wang, J., Li, Z., Luo, C., 2025. An anomaly detection model for in-vehicle networks based on lightweight convolution with spectral residuals. Comput. Secur. 151, 104304.

Miller, C., Valasek, C., 2015. Remote exploitation of an unaltered passenger vehicle. In: Black Hat USA, vol. 2015, (S 91), pp. 1–91.

Müter, M., Asaj, N., 2011. Entropy-based anomaly detection for in-vehicle networks. In: 2011 IEEE Intelligent Vehicles Symposium. IV, IEEE, pp. 1110–1115.

Navet, N., Song, Y., Simonot-Lion, F., Wilwert, C., 2005. Trends in automotive communication systems. Proc. IEEE 93 (6), 1204–1223.

Nguyen, T.P., Nam, H., Kim, D., 2023. Transformer-based attention network for in-vehicle intrusion detection. IEEE Access 11, 55389–55403.

Rosay, A., Riou, K., Carlier, F., Leroux, P., 2022. Multi-layer perceptron for network intrusion detection: From a study on two recent data sets to deployment on automotive processor. Ann. Telecommun. 77 (5), 371–394.

Sami, M., Ibarra, M., Esparza, A.C., Al-Jufout, S., Aliasgari, M., Mozumdar, M., 2020. Rapid, multi-vehicle and feed-forward neural network based intrusion detection system for controller area network bus. In: 2020 IEEE Green Energy and Smart Systems Conference. IGESSC, IEEE, pp. 1–6.

Song, H.M., Woo, J., Kim, H.K., 2020. In-vehicle network intrusion detection using deep convolutional neural network. Veh. Commun. 21, 100198.

Sun, H., Huang, W., Weng, J., Liu, Z., Tan, W., He, Z., Chen, M., Wu, B., Li, L., Peng, X., 2024. CCID-CAN: Cross-chain intrusion detection on CAN bus for autonomous vehicles. IEEE Internet Things J. 11 (15), 26146–26159.

Taneja, A., Kumar, G., 2024. Attention-CNN-LSTM based intrusion detection system (ACL-IDS) for in-vehicle networks: A. Taneja, g. Kumary. Soft Comput. 28 (23), 13429–13441.

Taylor, A., Japkowicz, N., Leblanc, S., 2015. Frequency-based anomaly detection for the automotive CAN bus. In: 2015 World Congress on Industrial Control Systems Security. WCICSS, IEEE, pp. 45–49.

Taylor, A., Leblanc, S., Japkowicz, N., 2016. Anomaly detection in automobile control network data with long short-term memory networks. In: 2016 IEEE International Conference on Data Science and Advanced Analytics. DSAA, IEEE, pp. 130–139.

Verma, M.E., Bridges, R.A., Iannacone, M.D., Hollifield, S.C., Moriano, P., Hespeler, S.C., Kay, B., Combs, F.L., 2024. A comprehensive guide to CAN IDS data and introduction of the ROAD dataset. PLoS ONE 19 (1), e0296879.

Wei, P., Wang, B., Dai, X., Li, L., He, F., 2023. A novel intrusion detection model for the CAN bus packet of in-vehicle network based on attention mechanism and autoencoder. Digit. Commun. Netw. 9 (1), 14–21.

Woo, S., Park, J., Lee, J.-Y., Kweon, I.S., 2018. Cbam: Convolutional block attention module. In: Proceedings of the European Conference on Computer Vision. ECCV, pp. 3–19.

Wu, W., Li, R., Xie, G., An, J., Bai, Y., Zhou, J., Li, K., 2019. A survey of intrusion detection for in-vehicle networks. IEEE Trans. Intell. Transp. Syst. 21 (3), 919–933.

Yin, Z., Li, Z., Zhang, Y., Li, J., Liu, D., Wei, H., 2025. Temporal-spatial feature fusion based intrusion detection system for in-vehicle networks. Comput. Secur. 104781.

Zhao, W., Yang, Y., Hu, H., Chen, Y., Yu, F., 2025. A lightweight intrusion detection approach for CAN bus using depthwise separable convolutional Kolmogorov arnold network. Sci. Rep. 15 (1), 17550.